

Homework - Neural networks - Part A (35 points)

Interactive activation and competition

by *Brenden Lake*

Note:

Part A is short answer without any code. Please complete the responses to these questions as a markdown cell inserted beneath the question prompts.

In this assignment, you will get hands on experience with a classic neural network model of memory known as the Interactive Activation and Competition (IAC) model. We will go through a series of exercises that will stretch your understanding of the IAC model in various ways. The exercises below examine how the mechanisms of interactive activation and competition can be used to illustrate two key properties of human memory:

- Retrieval by name and by content.
- Spontaneous generalization over a set of familiar items.

These exercises are from Chapter 2 of the [online PDP Handbook](#) by James McClelland.

You should review the slides from lecture and read Section 2.1 of the PDP Handbook before continuing. This has important background and technical details on how the IAC model works. The IAC model instantiates knowledge that someone may have from watching the 1960s musical "West Side Story," where two gangs the "Jets" and "Sharks" struggle for neighborhood control in Manhattan. The "database" for this exercise is the Jets and Sharks data base shown in Figure 1, which has the central characters from the two gangs. You are to use the IAC model in conjunction with this data base to run illustrative simulations of these basic properties of memory.

The Jets and The Sharks

Name	Gang	Age	Edu	Mar	Occupation
Art	Jets	40's	J.H.	Sing.	Pusher
Al	Jets	30's	J.H.	Mar.	Burglar
Sam	Jets	20's	COL.	Sing.	Bookie
Clyde	Jets	40's	J.H.	Sing.	Bookie
Mike	Jets	30's	J.H.	Sing.	Bookie
Jim	Jets	20's	J.H.	Div.	Burglar
Greg	Jets	20's	H.S.	Mar.	Pusher
John	Jets	20's	J.H.	Mar.	Burglar
Doug	Jets	30's	H.S.	Sing.	Bookie
Lance	Jets	20's	J.H.	Mar.	Burglar
George	Jets	20's	J.H.	Div.	Burglar
Pete	Jets	20's	H.S.	Sing.	Bookie
Fred	Jets	20's	H.S.	Sing.	Pusher
Gene	Jets	20's	COL.	Sing.	Pusher
Ralph	Jets	30's	J.H.	Sing.	Pusher
Phil	Sharks	30's	COL.	Mar.	Pusher
Ike	Sharks	30's	J.H.	Sing.	Bookie
Nick	Sharks	30's	H.S.	Sing.	Pusher
Don	Sharks	30's	COL.	Mar.	Burglar
Ned	Sharks	30's	COL.	Mar.	Bookie
Karl	Sharks	40's	H.S.	Mar.	Bookie
Ken	Sharks	20's	H.S.	Sing.	Burglar
Earl	Sharks	40's	H.S.	Mar.	Burglar
Rick	Sharks	30's	H.S.	Div.	Burglar
Ol	Sharks	30's	COL.	Mar.	Pusher
Neal	Sharks	30's	H.S.	Sing.	Bookie
Dave	Sharks	30's	H.S.	Div.	Pusher

Figure 1: Characteristics of a number of individuals belonging to two gangs, the Jets and the Sharks. (From "Retrieving General and Specific Knowledge From Stored Knowledge of Specifics" by J. L. McClelland, 1981, Proceedings of the Third Annual Conference of the Cognitive Science Society.)

Software and architecture

We will be using IAC software from Todd Gureckis which you access [here](#).

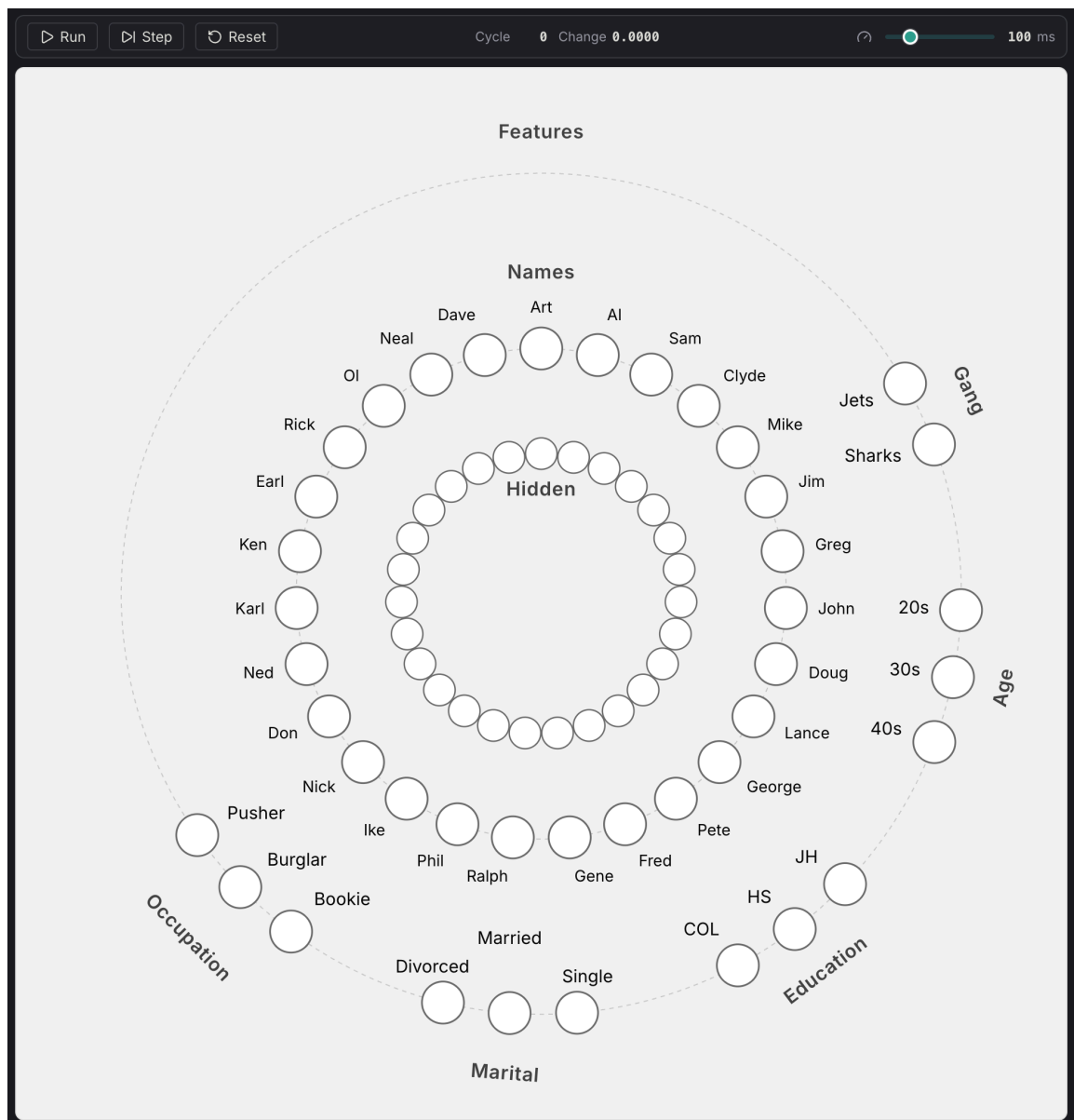


Figure 2: Screen shot from Gureckis's software. Units are organized into 7 groups.

You will see a display that looks like Figure 2. The units are grouped into seven pools: a pool of *visible* name units, a pool of *gang* units, a pool of *age* units, a pool of *education* units, a pool of *marital status* units, a pool of *occupation* units, and a pool of *hidden* units. The name pool contains a unit for the name of each person; the gang pool contains a unit for each of the gangs the people are members of (Jets and Sharks); the age pool contains a unit for each age range; and so on. Finally, the *hidden* pool contains an instance unit for each individual in the set.

The units in the first six pools can be called visible units, since all are assumed to be accessible from outside the network. Those in the gang, age, education, marital status, and occupation pools can also be called property units. The instance units are assumed to be inaccessible, so they can be called hidden units.

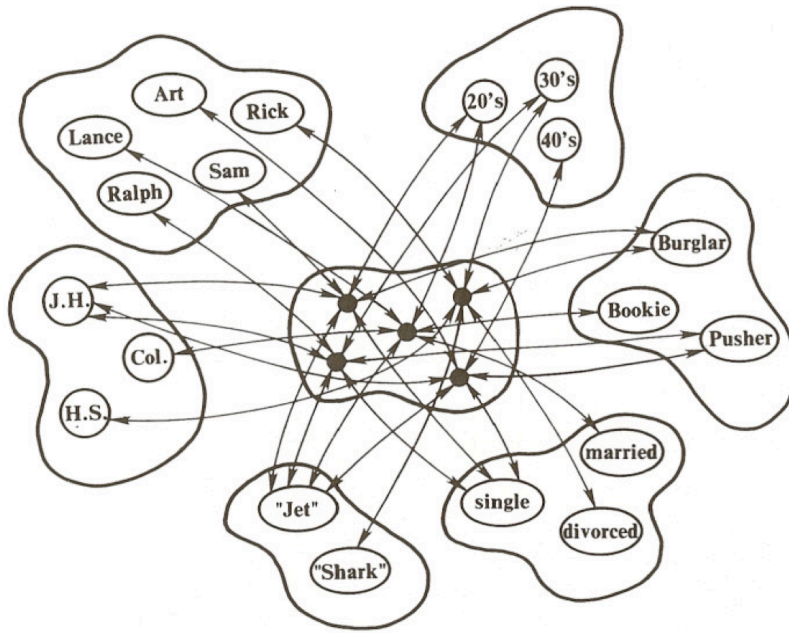


Figure 3: The units and connections for some of the individuals in Figure 1. The arrows represent excitatory connections. The outlined groups of units have mutually inhibitory connections (not shown). (From "Retrieving General and Specific Knowledge From Stored Knowledge of Specifics" by J. L. McClelland, 1981, *Proceedings of the Third Annual Conference of the Cognitive Science Society*.)

Each unit has an inhibitory connection to every other unit in the same pool. In addition, there are two-way excitatory connections between each instance unit and the units for its properties, as illustrated in Figure 3. Note that the figure is incomplete, in that only some of the name and instance units are shown. These names are given only for the convenience of the user, of course; all actual computation in the network occurs only by way of the connections. You can also view the different connections using the IAC software by hovering your mouse over a particular unit (Figure 4).

(Or click from the menu on top.) Set the external input of Ken's name unit to 1 by clicking on the name unit (not the hidden unit!). The circle should be highlighted in yellow to indicate the external input.

A unit's activation level can be visualized by the color. The baseline activation of -0.1 is shown as white. Higher activation than baseline is shown as pinkish, and lower activation than baseline is shown as blueish. A unit's precise activation can be examined by rolling the mouse over the unit.

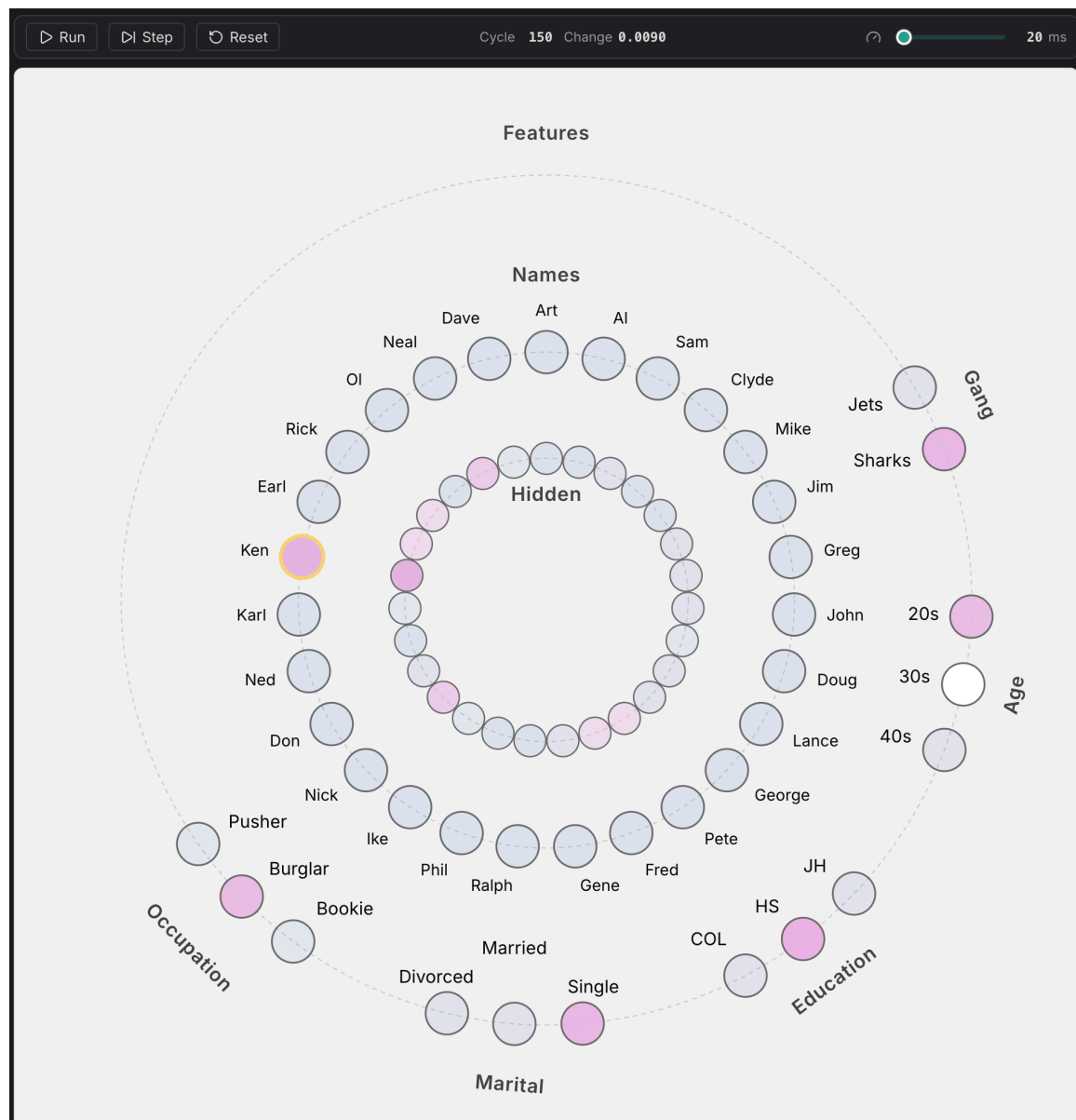


Figure 5: The display screen after about 150 cycles with external input to the name unit for Ken.

Press SPACE to unpause and allow the network to run for approximately 150 cycles (the cycle counter is at the top of the panel). Be sure to pause at about 150 cycles! You can also click Step for more granularity. A picture of the screen after 150 cycles is shown in Figure 5. At this point, you can check to see that the model has indeed retrieved the

pattern for Ken correctly. There are also several other things going on that are worth understanding. Answer all of the following questions below regarding the network at this state (you'll have to refer to the properties of the individuals, as given in Figure 1).

Problem 1 (10 points)

None of the visible name units other than Ken were activated, yet a few other hidden instance units are active (i.e., their activation is greater than 0). Explain why these units are active. Keep your response short (about 3 sentences).

Answer: Since Ken's instance units activate his property units ie the attributes that are fully lit up, those units send excitation back to every instance unit connected to them, not just Ken's. The people who share the most properties with Ken get enough of this feedback which is shown as the levels rise to above 0. This is shown as Nick, Neal, Rick, Pete and Fred have all levels above 0. This makes sense since Nick and Neal share both Sharks, HS, and Single, whereas Rick is a Shark, HS, and Burglur. No other name units turn on because Ken's instance unit inhibits the others, so they stay only partly active and send little activation out to their names.

Problem 2 (10 points)

Some of Ken's properties are activated more strongly than others. Why? Keep your response short (about 3 sentences).

Each property unit gets inputs from Ken's instance plus any any other active instances connected to it, ie those it has in common with others. Sharks and HS are shared by all three similar people (Nick, Neal, Rick), so they get most support. Single (Nick and Neal) and Burglur (Rick) get less. 20s is the weakest since none of the other active people are in their 20s. All three are in their 30s, so the 30s unit rises.

Retrieval from a partial description

Next, we will use the IAC software to illustrate how it can retrieve an instance from a partial description of its properties. We will continue to use Ken, who, as it happens, can be uniquely described by two properties, Shark and in 20s. Reset the network ('r') and make sure everything is paused and that all units have input of 0. Click to set the external input of the Sharks unit and the in 20s unit to 1.00. Run a total of 150 cycles again, and take a look at the state of the network.

Of all of the visible name units, Ken's name should be the most active. Compare this state of the network with the screen shot of the previous state of the network when activating Ken's name directly, such as that in Figure 5.

Problem 3 (10 points)

Explain why the occupation units show partial activations of units other than Ken's occupation, which is Burglar. While being succinct, try to get to the bottom of this, and contrast the current case with the previous case. Keep your response short (about 3 sentences).

Without the name unit, Ken's instance has no unique input on its own. It is just the one person who matches both probes, so instances matching one probe plus the properties Ken turns on (HS, Single) stay fairly active. These are Nick and Neal (Sharks) and Pete and Fred (20s). Those four are Pushers (Nick, Fred) and Bookies (Neal, Pete), and none are Burglurs. So Pusher and Bookie each get input from two instances, which nearly matches Burglur's input from Ken alone. When the name was the input, Ken's instance dominated more strongly. The similar people then split across occupations, so Burglur easily beat the others and pushed them below baseline.

Spontaneous generalization

Now we consider the network's ability to retrieve appropriate generalizations over sets of individuals—that is, its ability to answer questions like “What are Jets like?” or “What are people who are in their 20s and have only a junior high education like?” Reset ('r') the network. Set the external input of Jets to 1.00 by clicking on it. Run the network for 150 cycles and observe what happens.

Problem 4 (5 points)

Given the network's state, what can you infer about a typical Jet? (1-2 sentences is plenty).

A typical Jet is in his 20s, has a junior high education, and is single. Those units become strongly active even though no single Jet was probed, because most Jets have them (9 of 15 for each). The occupation units stay weak and about equal, because the Jets are split evenly from 5 Pushers, 5 Burglars and 5 Bookies hence the network doesn't predict a typical job.

Homework - Neural networks - Part B (55 points)

Gradient descent for simple two and three layer models

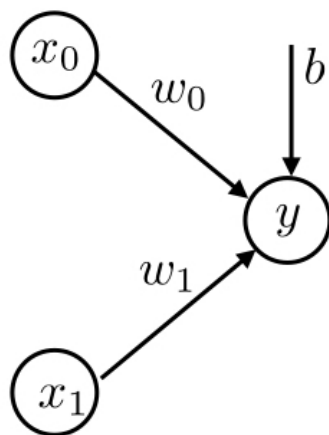
by Brenden Lake

The first part of this assignment implements the gradient descent algorithm for a simple artificial neuron. The second part implements backpropagation for a simple network with one hidden unit. Remember that we strongly suggest you *TURN OFF* any autocompletion functionality (e.g., in VSCode and Cursor) so that you can learn and think as you do this assignment. For instance, in VSCode, edit `settings.json` (on mac search via command-P) such that `quickSuggestions` are off.

```
"editor.quickSuggestions": {
  "other": "off",
  "comments": "off",
  "strings": "off"
}
```

Alternatively, working with Jupyter notebooks in the terminal also removes this temptation.

In the first part, the neuron will learn to compute logical OR. The neuron model and logical OR are shown below, for inputs x_0 and x_1 and target output y .



logical OR

x_0	x_1	y
0	0	0
0	1	1
1	0	1
1	1	1

This assignment requires some basic PyTorch knowledge. You can review your notes from lab and this [PyTorch tutorial](#). The "Introduction to PyTorch" section on the PyTorch website is also helpful.

```
In [1]: # Import libraries
from __future__ import print_function
%matplotlib inline
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import torch
```

Let's create `torch.tensor` objects for representing the data matrix `X` with targets `Y_or` (for the logical OR function). Each row of `X` is a different input pattern.

```
In [3]: X_list = [[0.,0.], [0.,1.], [1.,0.], [1.,1.]]
X = torch.tensor(X_list)
Y_or = torch.tensor([0.,1.,1.,1.])
N = X.shape[0] # number of input patterns
print("Input tensor X:")
print('  has shape',X.shape)
print('  and contains',X)
print('Target tensor Y:')
print('  has shape',Y_or.shape)
print('  and contains',Y_or)
```

```
Input tensor X:
  has shape torch.Size([4, 2])
  and contains tensor([[0., 0.],
                      [0., 1.],
                      [1., 0.],
                      [1., 1.]])
Target tensor Y:
  has shape torch.Size([4])
  and contains tensor([0., 1., 1., 1.])
```

The artificial neuron operates as follows. Given an input vector x (which is one row of input tensor X), the net input (**net**) to the neuron is computed as follows

$$\mathbf{net} = \sum_i x_i w_i + b,$$

for weights w_i and bias b . The activation function $g(\mathbf{net})$ is the logistic function

$$g(\mathbf{net}) = \frac{1}{1 + e^{-\mathbf{net}}},$$

which is used to compute the predicted output $\hat{y} = g(\mathbf{net})$. Finally, the loss (squared error) for a particular pattern x is defined as

$$E(w, b) = (\hat{y} - y)^2,$$

where the target output is y . **Your main task is to manually compute the partial derivatives of the loss E with respect to the neuron parameters:**

$$\frac{\partial E(w, b)}{\partial w}, \frac{\partial E(w, b)}{\partial b}.$$

By manually, we mean to program the gradient computation directly, using the formulas discussed in class. This is in contrast to using PyTorch's `autograd` (Automatic differentiation) that computes the gradient automatically, as discussed in class, lab, and in the PyTorch tutorial (e.g., `loss.backward()`). First, let's write the activation function and the loss in PyTorch.

```
In [4]: def g_logistic(net):  
        return 1. / (1.+torch.exp(-net))  
  
def loss(yhat,y):  
    return (yhat-y)**2
```

Next, we'll also write two functions for examining the internal operations of the neuron, and the gradients of its parameters.

```
In [5]: def print_forward(x,yhat,y):  
        # Examine network's prediction for input x  
        print(' Input: ',end='')  
        print(x.numpy())  
        print(' Output: ' + str(round(yhat.item(),3)))  
        print(' Target: ' + str(y.item()))  
  
def print_grad(grad_w,grad_b):  
    # Examine gradients  
    print(' d_loss / d_w = ',end='')  
    print(grad_w)  
    print(' d_loss / d_b = ',end='')  
    print(grad_b)
```

Now let's dive in and begin the implementation of stochastic gradient descent. We'll initialize our parameters w and b randomly, and proceed through a series of epochs of training. Each epoch involves visiting the four training patterns in random order, and updating the parameters after each presentation of an input pattern.

Problem 1 (10 points)

In the code below, fill in code to manually compute the gradient in closed form.

- See lecture slides for the equation for the gradient for the weights w .
- Derive (or reason) to get the equation for the gradient for bias b .

Problem 2 (5 points)

In the code below, fill in code for the weight and bias update rule for gradient

descent.

After completing the code, run it to compare **your gradients** with the **ground-truth computed by PyTorch**. (There may be small differences that you shouldn't worry about, e.g. within $1e-6$). Also, you can check the neuron's performance at the end of training.

```
In [6]: # Initialize parameters
#       Although you will implement gradient descent manually, let's set requires_grad=True
#       anyway so PyTorch will track the gradient too, and we can compare your
w = torch.randn(2, requires_grad=True) # [size 2] tensor
b = torch.randn(1, requires_grad=True) # [size 1] tensor

alpha = 0.05 # learning rate
nepochs = 5000 # number of epochs

track_error = []
verbose = True
for e in range(nepochs): # for each epoch
    error_epoch = 0. # sum loss across the epoch
    perm = np.random.permutation(N)
    for p in perm: # visit data points in random order
        x_pat = X[p,:] # get one input pattern

        # compute output of neuron
        net = torch.dot(x_pat,w)+b
        yhat = g_logistic(net)

        # compute loss
        y = Y_or[p]
        myloss = loss(yhat,y)
        error_epoch += myloss.item()

    # Compute the gradient manually
    if verbose:
        print('Compute the gradient manually')
        print_forward(x_pat,yhat,y)
    with torch.no_grad():
        # TODO : YOUR GRADIENT CODE GOES HERE
        # two lines of the form
        # w_grad = ... ([size 2] PyTorch tensor)
        # b_grad = ... ([size 1] PyTorch tensor)
        # make sure to enclose your code in the "with torch.no_grad()"
        # otherwise PyTorch will try to track the "gradient" of the gr
        w_grad = 2*(yhat - y)*yhat*(1 - yhat) * x_pat # dE/dyhat * dyhat
        b_grad = 2*(yhat - y)*yhat*(1 - yhat)          # dE/dyhat * dyhat
    if verbose: print_grad(w_grad.numpy(),b_grad.numpy())

    # Compute the gradient with PyTorch and compare with manual values
    if verbose: print('Compute the gradient using PyTorch .backward()')
    myloss.backward()
    if verbose:
        print_grad(w.grad.numpy(),b.grad.numpy())
        print('')
    w.grad.zero_() # clear PyTorch's gradient
```

```

b.grad.zero_()

# Parameter update with gradient descent
with torch.no_grad():
    # TODO : YOUR PARAMETER UPDATE CODE GOES HERE
    # two lines of the form:
    # w -= ....
    # b -= ....
    w -= alpha * w_grad
    b -= alpha * b_grad

if verbose==True: verbose=False
track_error.append(error_epoch)
if e % 50 == 0:
    print("epoch " + str(e) + "; error=" +str(round(error_epoch,3)))

# print a final pass through patterns
for p in range(X.shape[0]):
    x_pat = X[p]
    net = torch.dot(x_pat,w)+b
    yhat = g_logistic(net)
    y = Y_or[p]
    print("Final result:")
    print_forward(x_pat,yhat,y)
    print("")

# track output of gradient descent
plt.figure()
plt.clf()
plt.plot(track_error)
plt.title('stochastic gradient descent (logistic activation)')
plt.ylabel('error for epoch')
plt.xlabel('epoch')
plt.show()

```

```
Compute the gradient manually
Input: [1. 0.]
Output: 0.346
Target: 1.0
d_loss / d_w = [-0.29599196 -0.          ]
d_loss / d_b = [-0.29599196]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [-0.29599196 -0.          ]
d_loss / d_b = [-0.29599196]
```

```
Compute the gradient manually
Input: [1. 1.]
Output: 0.289
Target: 1.0
d_loss / d_w = [-0.29213744 -0.29213744]
d_loss / d_b = [-0.29213744]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [-0.29213747 -0.29213747]
d_loss / d_b = [-0.29213747]
```

```
Compute the gradient manually
Input: [0. 1.]
Output: 0.081
Target: 1.0
d_loss / d_w = [-0.          -0.13688438]
d_loss / d_b = [-0.13688438]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [ 0.          -0.13688436]
d_loss / d_b = [-0.13688436]
```

```
Compute the gradient manually
Input: [0. 0.]
Output: 0.105
Target: 0.0
d_loss / d_w = [0. 0.]
d_loss / d_b = [0.01974555]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [0. 0.]
d_loss / d_b = [0.01974555]
```

```
epoch 0; error=1.789
epoch 50; error=0.456
epoch 100; error=0.267
epoch 150; error=0.218
epoch 200; error=0.188
epoch 250; error=0.166
epoch 300; error=0.149
epoch 350; error=0.135
epoch 400; error=0.123
epoch 450; error=0.113
epoch 500; error=0.104
epoch 550; error=0.097
epoch 600; error=0.09
epoch 650; error=0.085
epoch 700; error=0.079
epoch 750; error=0.075
```

epoch 800; error=0.071
epoch 850; error=0.067
epoch 900; error=0.064
epoch 950; error=0.061
epoch 1000; error=0.058
epoch 1050; error=0.055
epoch 1100; error=0.053
epoch 1150; error=0.051
epoch 1200; error=0.049
epoch 1250; error=0.047
epoch 1300; error=0.045
epoch 1350; error=0.043
epoch 1400; error=0.042
epoch 1450; error=0.04
epoch 1500; error=0.039
epoch 1550; error=0.038
epoch 1600; error=0.037
epoch 1650; error=0.035
epoch 1700; error=0.034
epoch 1750; error=0.033
epoch 1800; error=0.032
epoch 1850; error=0.032
epoch 1900; error=0.031
epoch 1950; error=0.03
epoch 2000; error=0.029
epoch 2050; error=0.028
epoch 2100; error=0.028
epoch 2150; error=0.027
epoch 2200; error=0.026
epoch 2250; error=0.026
epoch 2300; error=0.025
epoch 2350; error=0.025
epoch 2400; error=0.024
epoch 2450; error=0.024
epoch 2500; error=0.023
epoch 2550; error=0.023
epoch 2600; error=0.022
epoch 2650; error=0.022
epoch 2700; error=0.021
epoch 2750; error=0.021
epoch 2800; error=0.02
epoch 2850; error=0.02
epoch 2900; error=0.02
epoch 2950; error=0.019
epoch 3000; error=0.019
epoch 3050; error=0.019
epoch 3100; error=0.018
epoch 3150; error=0.018
epoch 3200; error=0.018
epoch 3250; error=0.017
epoch 3300; error=0.017
epoch 3350; error=0.017
epoch 3400; error=0.017
epoch 3450; error=0.016
epoch 3500; error=0.016
epoch 3550; error=0.016

epoch 3600; error=0.016
epoch 3650; error=0.015
epoch 3700; error=0.015
epoch 3750; error=0.015
epoch 3800; error=0.015
epoch 3850; error=0.015
epoch 3900; error=0.014
epoch 3950; error=0.014
epoch 4000; error=0.014
epoch 4050; error=0.014
epoch 4100; error=0.014
epoch 4150; error=0.013
epoch 4200; error=0.013
epoch 4250; error=0.013
epoch 4300; error=0.013
epoch 4350; error=0.013
epoch 4400; error=0.013
epoch 4450; error=0.013
epoch 4500; error=0.012
epoch 4550; error=0.012
epoch 4600; error=0.012
epoch 4650; error=0.012
epoch 4700; error=0.012
epoch 4750; error=0.012
epoch 4800; error=0.012
epoch 4850; error=0.011
epoch 4900; error=0.011
epoch 4950; error=0.011

Final result:

Input: [0. 0.]
Output: 0.079
Target: 0.0

Final result:

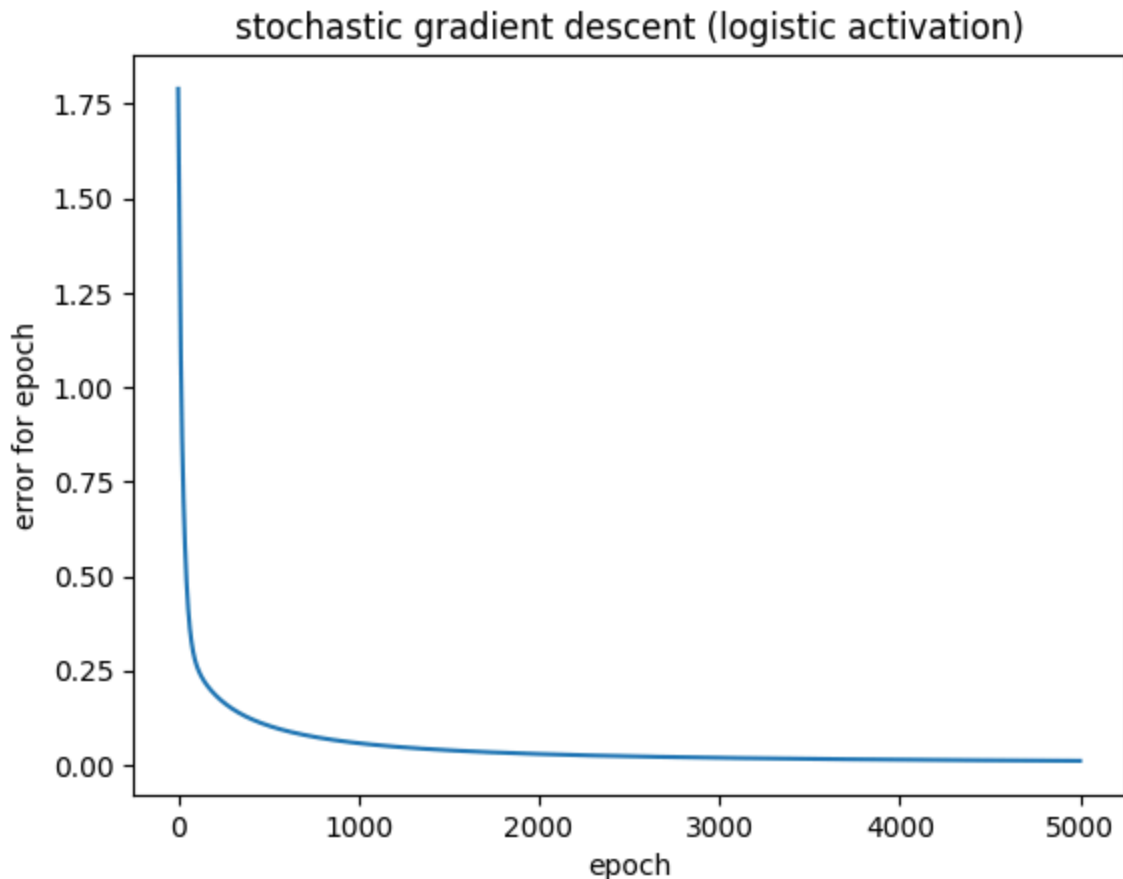
Input: [0. 1.]
Output: 0.951
Target: 1.0

Final result:

Input: [1. 0.]
Output: 0.951
Target: 1.0

Final result:

Input: [1. 1.]
Output: 1.0
Target: 1.0



Now let's change the activation function to "linear" (identity function) from the "logistic" function, such that $g(\mathbf{net}) = \mathbf{net}$. With a linear rather than logistic activation, the output will no longer be constrained between 0 and 1. The artificial neuron will still try to solve the problem with 0/1 targets. Here is the simple implementation of $g(\cdot)$:

```
In [7]: def g_linear(x):
        return x
```

Problem 3 (5 points)

Just as before, fill in the missing code fragments for implementing gradient descent. This time we are using the linear activation function. Be sure to change your gradient calculation to reflect the new activation function.

```
In [8]: # Initialize parameters
        # Although you will implement gradient descent manually, let's set requires_grad=True
        # anyway so PyTorch will track the gradient too, and we can compare your
        w = torch.randn(2, requires_grad=True) # [size 2] tensor
        b = torch.randn(1, requires_grad=True) # [size 1] tensor

        alpha = 0.05 # learning rate
        nepochs = 5000 # number of epochs
```

```

track_error = []
verbose = True
for e in range(nepochs): # for each epoch
    error_epoch = 0. # sum loss across the epoch
    perm = np.random.permutation(N)
    for p in perm: # visit data points in random order
        x_pat = X[p,:] # get one input pattern

        # compute output of neuron
        net = torch.dot(x_pat,w)+b
        yhat = g_linear(net)

        # compute loss
        y = Y_or[p]
        myloss = loss(yhat,y)
        error_epoch += myloss.item()

        # Compute the gradient manually
        if verbose:
            print('Compute the gradient manually')
            print_forward(x_pat,yhat,y)
        with torch.no_grad():
            # TODO : YOUR GRADIENT CODE GOES HERE
            # two lines of the form
            # w_grad = ... ([size 2] PyTorch tensor)
            # b_grad = ... ([size 1] PyTorch tensor)
            # make sure to enclose your code in the "with torch.no_grad()"
            # otherwise PyTorch will try to track the "gradient" of the gr
            w_grad = 2*(yhat - y) * x_pat # dE/dyhat * dyhat/dnet (=1 for li
            b_grad = 2*(yhat - y) # dE/dyhat * dyhat/dnet (=1 for li
        if verbose: print_grad(w_grad.numpy(),b_grad.numpy())

        # Compute the gradient with PyTorch and compare with manual values
        if verbose: print('Compute the gradient using PyTorch .backward()')
        myloss.backward()
        if verbose:
            print_grad(w.grad.numpy(),b.grad.numpy())
            print('')
        w.grad.zero_() # clear PyTorch's gradient
        b.grad.zero_()

        # Parameter update with gradient descent
        with torch.no_grad():
            # TODO : YOUR PARAMETER UPDATE CODE GOES HERE
            # two lines of the form:
            # w -= ....
            # b -= ....
            w -= alpha * w_grad
            b -= alpha * b_grad

    if verbose==True: verbose=False
    track_error.append(error_epoch)
    if e % 50 == 0:
        print("epoch " + str(e) + "; error=" +str(round(error_epoch,3)))

```

```
# print a final pass through patterns
for p in range(X.shape[0]):
    x_pat = X[p]
    net = torch.dot(x_pat,w)+b
    yhat = g_linear(net)
    y = Y_or[p]
    print("Final result:")
    print_forward(x_pat,yhat,y)
    print("")

# track output of gradient descent
plt.figure()
plt.clf()
plt.plot(track_error)
plt.title('stochastic gradient descent (linear/null activation)')
plt.ylabel('error for epoch')
plt.xlabel('epoch')
plt.show()
```

```
Compute the gradient manually
Input: [1. 1.]
Output: -1.915
Target: 1.0
d_loss / d_w = [-5.8293734 -5.8293734]
d_loss / d_b = [-5.8293734]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [-5.8293734 -5.8293734]
d_loss / d_b = [-5.8293734]
```

```
Compute the gradient manually
Input: [0. 0.]
Output: -1.107
Target: 0.0
d_loss / d_w = [-0. -0.]
d_loss / d_b = [-2.2142534]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [0. 0.]
d_loss / d_b = [-2.2142534]
```

```
Compute the gradient manually
Input: [0. 1.]
Output: -1.789
Target: 1.0
d_loss / d_w = [-0. -5.5789356]
d_loss / d_b = [-5.5789356]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [ 0. -5.5789356]
d_loss / d_b = [-5.5789356]
```

```
Compute the gradient manually
Input: [1. 0.]
Output: 0.142
Target: 1.0
d_loss / d_w = [-1.7151347 -0. ]
d_loss / d_b = [-1.7151347]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [-1.7151347 0. ]
d_loss / d_b = [-1.7151347]
```

```
epoch 0; error=18.238
epoch 50; error=0.3
epoch 100; error=0.298
epoch 150; error=0.299
epoch 200; error=0.297
epoch 250; error=0.304
epoch 300; error=0.304
epoch 350; error=0.306
epoch 400; error=0.307
epoch 450; error=0.309
epoch 500; error=0.308
epoch 550; error=0.3
epoch 600; error=0.305
epoch 650; error=0.3
epoch 700; error=0.308
epoch 750; error=0.3
```

epoch 800; error=0.299
epoch 850; error=0.296
epoch 900; error=0.297
epoch 950; error=0.308
epoch 1000; error=0.301
epoch 1050; error=0.309
epoch 1100; error=0.301
epoch 1150; error=0.308
epoch 1200; error=0.308
epoch 1250; error=0.308
epoch 1300; error=0.307
epoch 1350; error=0.3
epoch 1400; error=0.308
epoch 1450; error=0.306
epoch 1500; error=0.307
epoch 1550; error=0.299
epoch 1600; error=0.307
epoch 1650; error=0.308
epoch 1700; error=0.301
epoch 1750; error=0.308
epoch 1800; error=0.307
epoch 1850; error=0.305
epoch 1900; error=0.306
epoch 1950; error=0.304
epoch 2000; error=0.308
epoch 2050; error=0.304
epoch 2100; error=0.303
epoch 2150; error=0.306
epoch 2200; error=0.304
epoch 2250; error=0.3
epoch 2300; error=0.297
epoch 2350; error=0.301
epoch 2400; error=0.306
epoch 2450; error=0.308
epoch 2500; error=0.307
epoch 2550; error=0.305
epoch 2600; error=0.301
epoch 2650; error=0.303
epoch 2700; error=0.306
epoch 2750; error=0.3
epoch 2800; error=0.309
epoch 2850; error=0.302
epoch 2900; error=0.308
epoch 2950; error=0.306
epoch 3000; error=0.306
epoch 3050; error=0.303
epoch 3100; error=0.302
epoch 3150; error=0.296
epoch 3200; error=0.303
epoch 3250; error=0.3
epoch 3300; error=0.306
epoch 3350; error=0.303
epoch 3400; error=0.308
epoch 3450; error=0.3
epoch 3500; error=0.306
epoch 3550; error=0.303

epoch 3600; error=0.306
epoch 3650; error=0.307
epoch 3700; error=0.296
epoch 3750; error=0.308
epoch 3800; error=0.308
epoch 3850; error=0.306
epoch 3900; error=0.303
epoch 3950; error=0.31
epoch 4000; error=0.308
epoch 4050; error=0.298
epoch 4100; error=0.304
epoch 4150; error=0.308
epoch 4200; error=0.305
epoch 4250; error=0.306
epoch 4300; error=0.307
epoch 4350; error=0.306
epoch 4400; error=0.31
epoch 4450; error=0.307
epoch 4500; error=0.3
epoch 4550; error=0.309
epoch 4600; error=0.3
epoch 4650; error=0.304
epoch 4700; error=0.306
epoch 4750; error=0.307
epoch 4800; error=0.306
epoch 4850; error=0.303
epoch 4900; error=0.31
epoch 4950; error=0.301

Final result:

Input: [0. 0.]

Output: 0.267

Target: 0.0

Final result:

Input: [0. 1.]

Output: 0.748

Target: 1.0

Final result:

Input: [1. 0.]

Output: 0.751

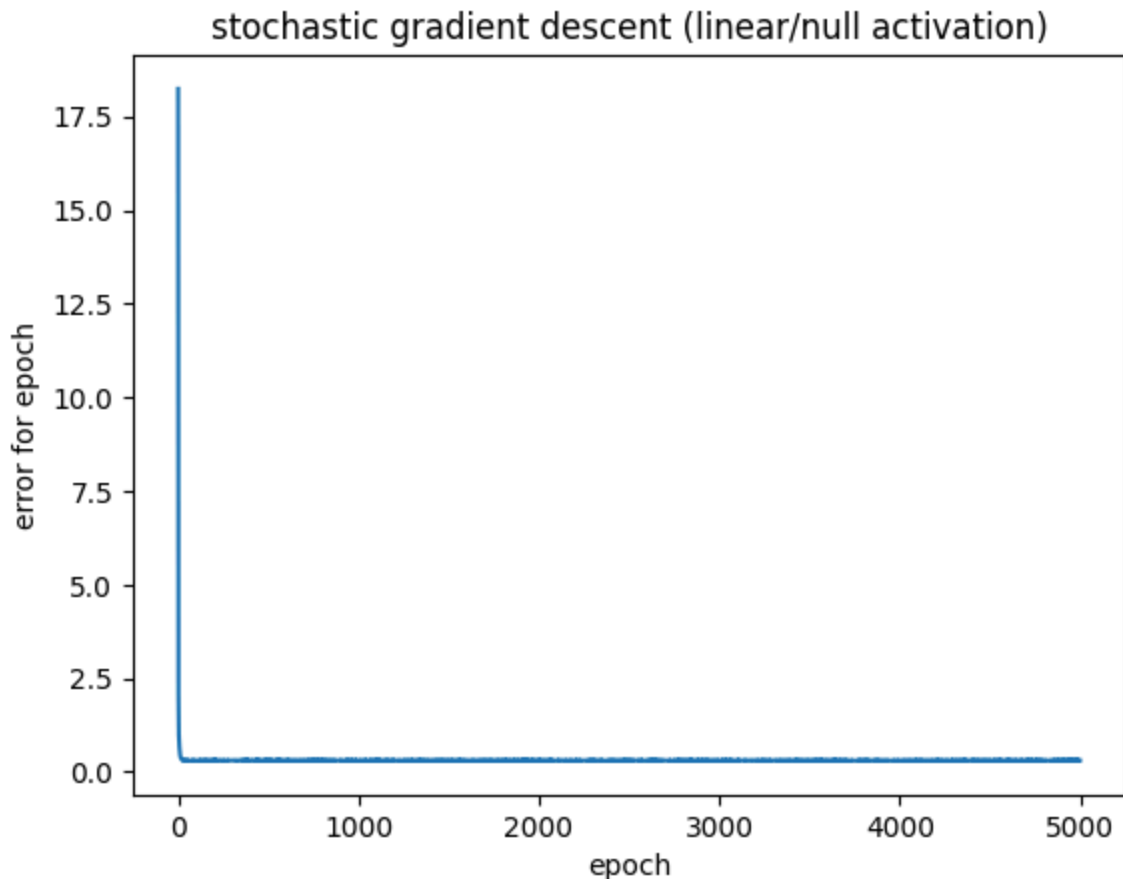
Target: 1.0

Final result:

Input: [1. 1.]

Output: 1.231

Target: 1.0



```
In [9]: # Learned parameters for the linear neuron
print('w =',w.data.numpy(), ' b =',b.item())
```

```
w = [0.48346823 0.4808231 ]  b = 0.2670857012271881
```

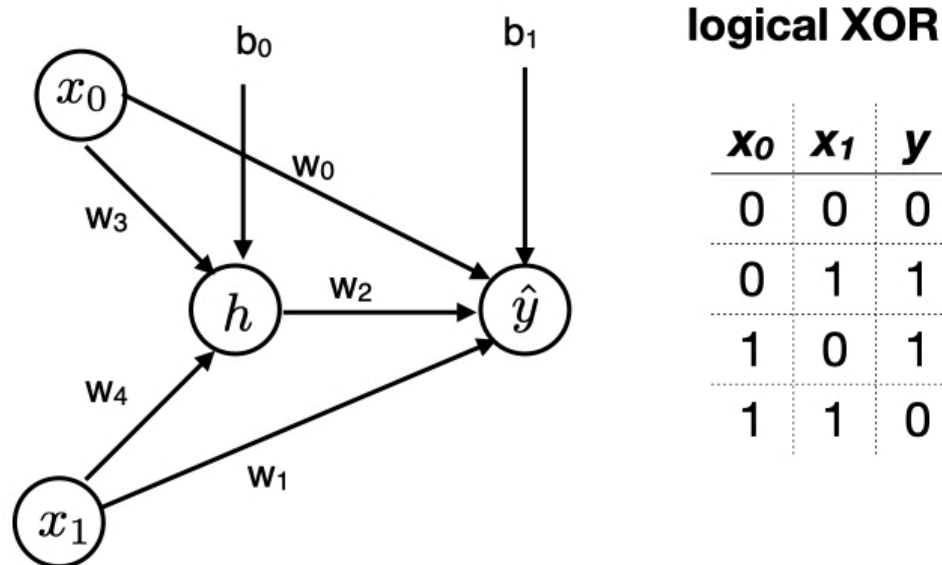
Problem 4 (10 points)

You'll see above that the artificial neuron, with the simple linear (identity) activation, does worse on the OR problem. Examine the learned weights and bias, and explain why the network does not arrive at a perfect solution.

Answer: The learned parameters are $w_0 \approx 0.48$, $w_1 \approx 0.48$, $b \approx 0.27$, giving outputs of about 0.27, 0.75, 0.75, and 1.23 for the four patterns (targets 0, 1, 1, 1). With a linear activation, the output is just $w_0x_0 + w_1x_1 + b$, so each active input adds a fixed amount and the effects simply sum; there is no squashing function to "saturate" at 1. OR needs the output to be the same (1) whether one or both inputs are on, but a linear unit cannot do this: if the weights were large enough (≈ 1) to bring $[0,1]$ and $[1,0]$ up to 1, then $[1,1]$ would output ≈ 2 , and the bias needed for $[0,0]$ to be 0 would also be in conflict. Since no linear function fits all four targets exactly, gradient descent instead finds the compromise that minimizes the total squared error, which is $w_0 = w_1 = 0.5$, $b = 0.25$ (outputs 0.25, 0.75, 0.75, 1.25, each off by 0.25). Our learned values hover around this

least-squares solution because stochastic updates keep nudging them. The logistic neuron avoids this problem because its output flattens out near 1, so a large net input from one or two active inputs produces nearly the same output.

In the next part, we have a simple multi-layer network with two input neurons, one hidden neuron, and one output neuron. Both the hidden and output unit should use the logistic activation function. We will learn to compute logical XOR. The network and logical XOR are shown below, for inputs x_0 and x_1 and target output y .



Problem 5 (15 points)

You will implement backpropagation for this simple network. In the code below, you have several parts to fill in. First, define the forward pass to compute the output $\hat{y}$ from the input x . Second, fill in code to manually compute the gradients for all five weights w and two biases b in closed form. Third, fill in the code for updating the biases and weights.

After completing the code, run it to compare **your gradients** with the **ground-truth computed by PyTorch**. (There may be small differences that you shouldn't worry about, e.g. within $1e-6$). Also, you can check the network's performance at the end of training.

```
In [10]: # Same input tensor X and new labels y for xor
Y_xor = torch.tensor([0.,1.,1.,0.])
N = X.shape[0] # number of input patterns

# Initialize parameters
# Although you will implement gradient descent manually, let's set requires_grad_
# anyway so PyTorch will track the gradient too, and we can compare your
w_34 = torch.randn(2,requires_grad=True) # [size 2] tensor representing [w_3
```

```

w_012 = torch.randn(3,requires_grad=True) # [size 3] tensor representing [w_
b_0 = torch.randn(1,requires_grad=True) # [size 1] tensor
b_1 = torch.randn(1,requires_grad=True) # [size 1] tensor

alpha = 0.05 # learning rate
nepochs = 10000 # number of epochs

track_error = []
verbose = True
for e in range(nepochs): # for each epoch
    error_epoch = 0. # sum loss across the epoch
    perm = np.random.permutation(N)
    for p in perm: # visit data points in random order
        x_pat = X[p,:] # input pattern

        # Compute the output of hidden neuron h
        # e.g., two lines like the following
        # net_h = ...
        # h = ...
        # TODO : YOUR CODE GOES HERE
        net_h = torch.dot(x_pat,w_34) + b_0
        h = g_logistic(net_h)

        # Compute the output of neuron yhat
        # e.g., two lines like the following
        # net_y = ...
        # yhat = ...
        # TODO : YOUR CODE GOES HERE
        net_y = torch.dot(x_pat,w_012[:2]) + w_012[2]*h + b_1
        yhat = g_logistic(net_y)

        # compute loss
        y = Y_xor[p]
        myloss = loss(yhat,y)
        error_epoch += myloss.item()

    # print output if this is the last epoch
    if (e == nepochs-1):
        print("Final result:")
        print_forward(x_pat,yhat,y)
        print("")

    # Compute the gradient manually
    if verbose:
        print('Compute the gradient manually')
        print_forward(x_pat,yhat,y)
    with torch.no_grad():
        # TODO : YOUR GRADIENT CODE GOES HERE
        # should include at least these 4 lines (helper lines may be us
        # w_34_grad = ...
        # b_0_grad = ...
        # w_012_grad = ...
        # b_1_grad = ...
        # make sure to enclose your code in the "with torch.no_grad()"
        # otherwise PyTorch will try to track the "gradient" of the gr
        delta_y = 2*(yhat - y)*yhat*(1 - yhat) # error signal at output

```

```

        w_012_grad = delta_y * torch.cat((x_pat,h)) # inputs to output u
        b_1_grad = delta_y
        delta_h = delta_y * w_012[2] * h*(1 - h) # backpropagate through
        w_34_grad = delta_h * x_pat
        b_0_grad = delta_h
    if verbose:
        print(" Grad for w_34 and b_0")
        print_grad(w_34_grad.numpy(),b_0_grad.numpy())
        print(" Grad for w_012 and b_1")
        print_grad(w_012_grad.numpy(),b_1_grad.numpy())
        print("")

    # Compute the gradient with PyTorch and compare with manual values
    if verbose: print('Compute the gradient using PyTorch .backward()')
    myloss.backward()
    if verbose:
        print(" Grad for w_34 and b_0")
        print_grad(w_34.grad.numpy(),b_0.grad.numpy())
        print(" Grad for w_012 and b_1")
        print_grad(w_012.grad.numpy(),b_1.grad.numpy())
        print("")
    w_34.grad.zero_() # clear PyTorch's gradient
    b_0.grad.zero_()
    w_012.grad.zero_()
    b_1.grad.zero_()

    # Parameter update with gradient descent
    with torch.no_grad():
        # TODO : YOUR PARAMETER UPDATE CODE GOES HERE
        # Four lines of the form
        # w_34 -= ...
        # b_0 -= ...
        # w_012 -= ...
        # b_1 -= ...
        w_34 -= alpha * w_34_grad
        b_0 -= alpha * b_0_grad
        w_012 -= alpha * w_012_grad
        b_1 -= alpha * b_1_grad

    if verbose==True: verbose=False
    track_error.append(error_epoch)
    if e % 50 == 0:
        print("epoch " + str(e) + "; error=" +str(round(error_epoch,3)))

# track output of gradient descent
plt.figure()
plt.clf()
plt.plot(track_error)
plt.title('stochastic gradient descent (XOR)')
plt.ylabel('error for epoch')
plt.xlabel('epoch')
plt.show()

```

Compute the gradient manually

Input: [1. 0.]

Output: 0.124

Target: 1.0

Grad for w_34 and b_0

d_loss / d_w = [0.02602027 0.]

d_loss / d_b = [0.02602027]

Grad for w_012 and b_1

d_loss / d_w = [-0.18982297 -0. -0.12192427]

d_loss / d_b = [-0.18982297]

Compute the gradient using PyTorch .backward()

Grad for w_34 and b_0

d_loss / d_w = [0.02602027 0.]

d_loss / d_b = [0.02602027]

Grad for w_012 and b_1

d_loss / d_w = [-0.18982297 0. -0.12192427]

d_loss / d_b = [-0.18982297]

Compute the gradient manually

Input: [0. 0.]

Output: 0.237

Target: 0.0

Grad for w_34 and b_0

d_loss / d_w = [-0. -0.]

d_loss / d_b = [-0.01255374]

Grad for w_012 and b_1

d_loss / d_w = [0. 0. 0.04672741]

d_loss / d_b = [0.08572872]

Compute the gradient using PyTorch .backward()

Grad for w_34 and b_0

d_loss / d_w = [0. 0.]

d_loss / d_b = [-0.01255374]

Grad for w_012 and b_1

d_loss / d_w = [0. 0. 0.04672741]

d_loss / d_b = [0.08572872]

Compute the gradient manually

Input: [0. 1.]

Output: 0.156

Target: 1.0

Grad for w_34 and b_0

d_loss / d_w = [0. 0.02564379]

d_loss / d_b = [0.02564379]

Grad for w_012 and b_1

d_loss / d_w = [-0. -0.22198938 -0.16312972]

d_loss / d_b = [-0.22198938]

Compute the gradient using PyTorch .backward()

Grad for w_34 and b_0

d_loss / d_w = [0. 0.02564379]

d_loss / d_b = [0.02564379]

Grad for w_012 and b_1

d_loss / d_w = [0. -0.2219894 -0.16312972]

d_loss / d_b = [-0.2219894]

Compute the gradient manually

Input: [1. 1.]

Output: 0.082

Target: 0.0

Grad for w_34 and b_0

$d_{\text{loss}} / d_w = [-0.001137 \ -0.001137]$

$d_{\text{loss}} / d_b = [-0.001137]$

Grad for w_012 and b_1

$d_{\text{loss}} / d_w = [0.01239525 \ 0.01239525 \ 0.00998017]$

$d_{\text{loss}} / d_b = [0.01239525]$

Compute the gradient using PyTorch .backward()

Grad for w_34 and b_0

$d_{\text{loss}} / d_w = [-0.001137 \ -0.001137]$

$d_{\text{loss}} / d_b = [-0.001137]$

Grad for w_012 and b_1

$d_{\text{loss}} / d_w = [0.01239525 \ 0.01239525 \ 0.00998017]$

$d_{\text{loss}} / d_b = [0.01239525]$

epoch 0; error=1.544

epoch 50; error=1.039

epoch 100; error=1.017

epoch 150; error=1.016

epoch 200; error=1.016

epoch 250; error=1.016

epoch 300; error=1.015

epoch 350; error=1.015

epoch 400; error=1.015

epoch 450; error=1.015

epoch 500; error=1.015

epoch 550; error=1.015

epoch 600; error=1.015

epoch 650; error=1.015

epoch 700; error=1.015

epoch 750; error=1.014

epoch 800; error=1.014

epoch 850; error=1.014

epoch 900; error=1.014

epoch 950; error=1.014

epoch 1000; error=1.014

epoch 1050; error=1.014

epoch 1100; error=1.014

epoch 1150; error=1.014

epoch 1200; error=1.013

epoch 1250; error=1.013

epoch 1300; error=1.013

epoch 1350; error=1.013

epoch 1400; error=1.013

epoch 1450; error=1.013

epoch 1500; error=1.012

epoch 1550; error=1.012

epoch 1600; error=1.011

epoch 1650; error=1.011

epoch 1700; error=1.01

epoch 1750; error=1.01

epoch 1800; error=1.009
epoch 1850; error=1.008
epoch 1900; error=1.007
epoch 1950; error=1.006
epoch 2000; error=1.005
epoch 2050; error=1.004
epoch 2100; error=1.002
epoch 2150; error=1.0
epoch 2200; error=0.998
epoch 2250; error=0.995
epoch 2300; error=0.992
epoch 2350; error=0.989
epoch 2400; error=0.984
epoch 2450; error=0.98
epoch 2500; error=0.974
epoch 2550; error=0.968
epoch 2600; error=0.961
epoch 2650; error=0.953
epoch 2700; error=0.944
epoch 2750; error=0.933
epoch 2800; error=0.921
epoch 2850; error=0.907
epoch 2900; error=0.892
epoch 2950; error=0.874
epoch 3000; error=0.855
epoch 3050; error=0.833
epoch 3100; error=0.81
epoch 3150; error=0.785
epoch 3200; error=0.758
epoch 3250; error=0.73
epoch 3300; error=0.7
epoch 3350; error=0.67
epoch 3400; error=0.639
epoch 3450; error=0.609
epoch 3500; error=0.579
epoch 3550; error=0.549
epoch 3600; error=0.52
epoch 3650; error=0.493
epoch 3700; error=0.466
epoch 3750; error=0.441
epoch 3800; error=0.417
epoch 3850; error=0.395
epoch 3900; error=0.374
epoch 3950; error=0.354
epoch 4000; error=0.336
epoch 4050; error=0.318
epoch 4100; error=0.302
epoch 4150; error=0.287
epoch 4200; error=0.273
epoch 4250; error=0.26
epoch 4300; error=0.248
epoch 4350; error=0.237
epoch 4400; error=0.226
epoch 4450; error=0.217
epoch 4500; error=0.207
epoch 4550; error=0.199

epoch 4600; error=0.191
epoch 4650; error=0.183
epoch 4700; error=0.176
epoch 4750; error=0.169
epoch 4800; error=0.163
epoch 4850; error=0.157
epoch 4900; error=0.152
epoch 4950; error=0.147
epoch 5000; error=0.142
epoch 5050; error=0.137
epoch 5100; error=0.133
epoch 5150; error=0.128
epoch 5200; error=0.124
epoch 5250; error=0.121
epoch 5300; error=0.117
epoch 5350; error=0.114
epoch 5400; error=0.111
epoch 5450; error=0.107
epoch 5500; error=0.105
epoch 5550; error=0.102
epoch 5600; error=0.099
epoch 5650; error=0.097
epoch 5700; error=0.094
epoch 5750; error=0.092
epoch 5800; error=0.09
epoch 5850; error=0.088
epoch 5900; error=0.085
epoch 5950; error=0.084
epoch 6000; error=0.082
epoch 6050; error=0.08
epoch 6100; error=0.078
epoch 6150; error=0.076
epoch 6200; error=0.075
epoch 6250; error=0.073
epoch 6300; error=0.072
epoch 6350; error=0.07
epoch 6400; error=0.069
epoch 6450; error=0.068
epoch 6500; error=0.066
epoch 6550; error=0.065
epoch 6600; error=0.064
epoch 6650; error=0.063
epoch 6700; error=0.062
epoch 6750; error=0.061
epoch 6800; error=0.059
epoch 6850; error=0.058
epoch 6900; error=0.057
epoch 6950; error=0.057
epoch 7000; error=0.056
epoch 7050; error=0.055
epoch 7100; error=0.054
epoch 7150; error=0.053
epoch 7200; error=0.052
epoch 7250; error=0.051
epoch 7300; error=0.051
epoch 7350; error=0.05

epoch 7400; error=0.049
epoch 7450; error=0.048
epoch 7500; error=0.048
epoch 7550; error=0.047
epoch 7600; error=0.046
epoch 7650; error=0.046
epoch 7700; error=0.045
epoch 7750; error=0.044
epoch 7800; error=0.044
epoch 7850; error=0.043
epoch 7900; error=0.043
epoch 7950; error=0.042
epoch 8000; error=0.042
epoch 8050; error=0.041
epoch 8100; error=0.04
epoch 8150; error=0.04
epoch 8200; error=0.04
epoch 8250; error=0.039
epoch 8300; error=0.039
epoch 8350; error=0.038
epoch 8400; error=0.038
epoch 8450; error=0.037
epoch 8500; error=0.037
epoch 8550; error=0.036
epoch 8600; error=0.036
epoch 8650; error=0.036
epoch 8700; error=0.035
epoch 8750; error=0.035
epoch 8800; error=0.034
epoch 8850; error=0.034
epoch 8900; error=0.034
epoch 8950; error=0.033
epoch 9000; error=0.033
epoch 9050; error=0.033
epoch 9100; error=0.032
epoch 9150; error=0.032
epoch 9200; error=0.032
epoch 9250; error=0.031
epoch 9300; error=0.031
epoch 9350; error=0.031
epoch 9400; error=0.03
epoch 9450; error=0.03
epoch 9500; error=0.03
epoch 9550; error=0.029
epoch 9600; error=0.029
epoch 9650; error=0.029
epoch 9700; error=0.029
epoch 9750; error=0.028
epoch 9800; error=0.028
epoch 9850; error=0.028
epoch 9900; error=0.028
epoch 9950; error=0.027
Final result:

Input: [1. 0.]

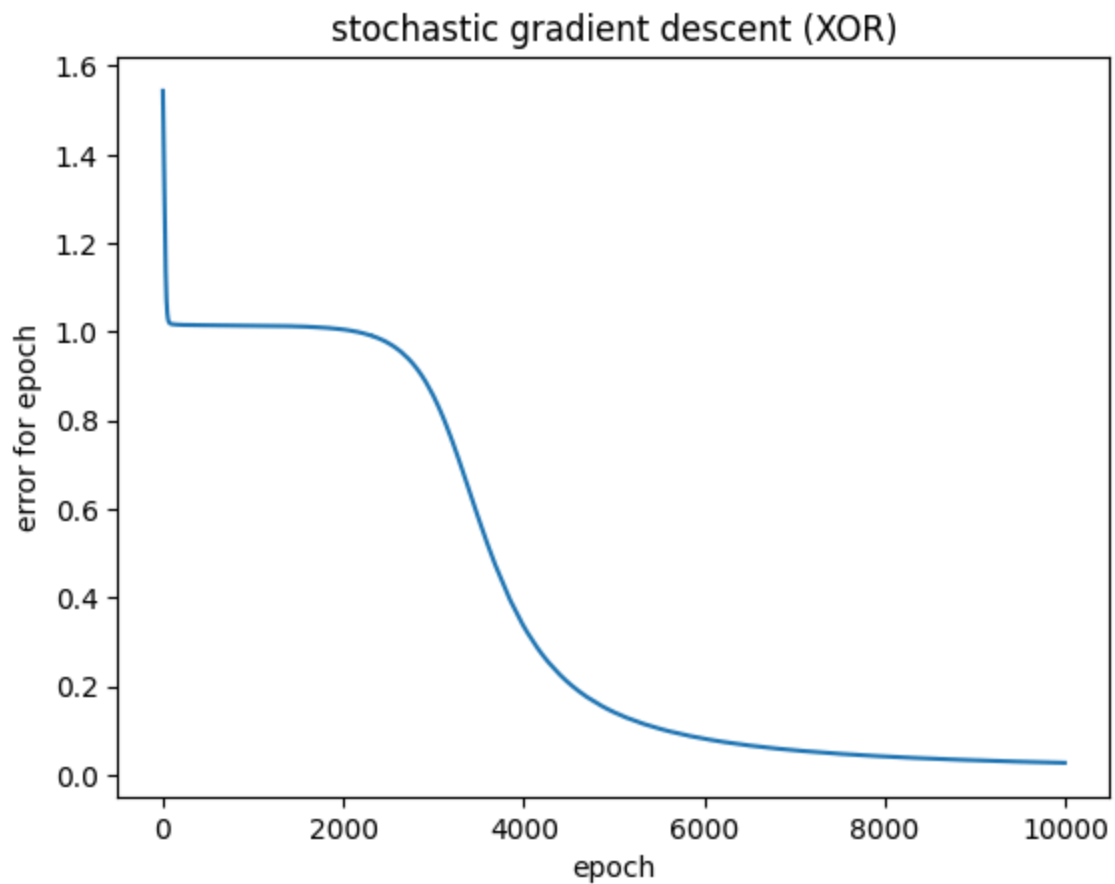
Output: 0.919

Target: 1.0

Final result:
Input: [0. 0.]
Output: 0.073
Target: 0.0

Final result:
Input: [0. 1.]
Output: 0.919
Target: 1.0

Final result:
Input: [1. 1.]
Output: 0.094
Target: 0.0



Problem 6 (10 points)

After running your XOR network, print the values of the learned weights and biases. Your job now is to describe the solution that the network has learned. How does it work? Walk through each input pattern to describe how the network computes the right answer (if it does). See discussion in lecture for an example.

```
In [11]: # Learned XOR parameters, and what each unit does for each input pattern
print('w_34 =',w_34.data.numpy(),' b_0 =',b_0.item())
print('w_012 =',w_012.data.numpy(),' b_1 =',b_1.item())
with torch.no_grad():
    for p in range(N):
        x_pat = X[p]
        net_h = torch.dot(x_pat,w_34) + b_0
        h = g_logistic(net_h)
        net_y = torch.dot(x_pat,w_012[:2]) + w_012[2]*h + b_1
        yhat = g_logistic(net_y)
        print(f'x={x_pat.numpy()} net_h={net_h.item():7.2f} h={h.item():.3f}
```

```
w_34 = [6.6760035  6.677774 ] b_0 = -2.728466749191284
w_012 = [-4.90621  -4.906645  10.74449 ] b_1 = -3.2046456336975098
x=[0. 0.] net_h= -2.73 h=0.061 net_y= -2.55 yhat=0.073 target=0
x=[0. 1.] net_h=  3.95 h=0.981 net_y=  2.43 yhat=0.919 target=1
x=[1. 0.] net_h=  3.95 h=0.981 net_y=  2.43 yhat=0.919 target=1
x=[1. 1.] net_h= 10.63 h=1.000 net_y= -2.27 yhat=0.093 target=0
```

Answer: Learned weights: hidden unit $w_3 \approx w_4 \approx 6.7$, $b_0 \approx -2.7$; output unit $w_0 \approx w_1 \approx -4.9$, $w_2 \approx 10.7$, $b_1 \approx -3.2$.

The hidden unit works like an **OR** meaning it turns on if at least one input is on. The output unit gets a big boost from the hidden unit but is pushed down by each input directly. One input on isn't enough to cancel the boost, but two inputs on is. So the network computes "OR, but not both", which is XOR.

- **[0,0]:** Hidden unit is off, so the output only gets its negative bias. Output ≈ 0.07 (target 0).
- **[0,1] and [1,0]:** Hidden unit turns on and boosts the output; one input pushes it down a little. Output ≈ 0.92 (target 1).
- **[1,1]:** Hidden unit is on, but both inputs push the output down, cancelling the boost. Output ≈ 0.09 (target 0).

So the network gets all four patterns right.

Homework - Neural networks - Part C (25 points)

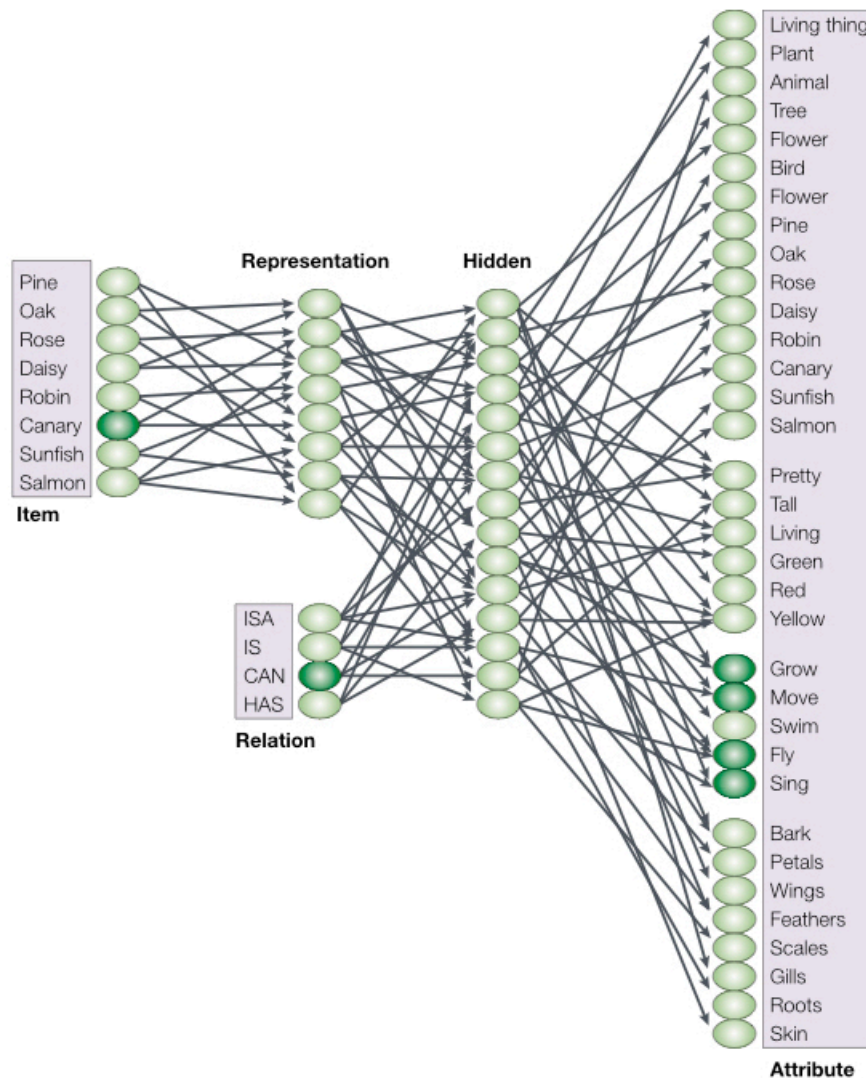
A neural network model of semantic cognition

by *Brenden Lake*

In this assignment, you will help implement and analyze a neural network model of semantic cognition. Semantic cognition is our intuitive understanding of objects and their properties. Semantic knowledge includes observations of which objects have which properties, and storage of these facts in long term memory. It also includes the ability to generalize, or predict which properties apply to which objects although they have not been directly observed.

This notebook explores a neural network model of semantic cognition developed by Rogers and McClelland (R&M). R&M sought to model aspects of semantic cognition with a multi-layer neural network, which contrasts with classic symbolic approaches for organizing semantic knowledge. They model the cognitive development of semantic representation as gradient descent (the backpropagation algorithm), using a neural network trained to map objects to their corresponding properties. R&M also modeled the deterioration of semantic knowledge in dementia by adding noise to the learned representations.

The network architecture is illustrated below.



There are two input layers ("Item Layer" and "Relation Layer"), which pass through intermediate layers to produce an output pattern on the "Attribute Layer." In this example, dark green is used to indicate active nodes (activation 1) and light green for inactive nodes (activation 0). The network is trained to answer queries involving an item (e.g., "Canary") and a relation (e.g., "CAN"), outputting all attributes that are true of the item/relation pair (e.g., "grow, move, fly, sing").

For this assignment, you will set up the network architecture in PyTorch and train it. The dataset and code for training has been provided. You will then analyze how its semantic knowledge develops over the course of training. While the original model used logistic (sigmoid) activation functions for all of the intermediate and output layers, we will use the ReLu activation for the Representation and Hidden Layers, with a sigmoid activation for the Attribute Layer.

Completing this assignment requires knowledge of setting up a neural network architecture in PyTorch. Please review your notes from lab and the following [PyTorch](#)

tutorial.

Reference (available for download on Canvas):

McClelland, J. L., & Rogers, T. T. (2003). The parallel distributed processing approach to semantic cognition. *Nature Reviews Neuroscience*, 4(4), 310.

```
In [1]: # Import libraries
        from __future__ import print_function
        import matplotlib
        %matplotlib inline
        import matplotlib.pyplot as plt
        import numpy as np
        import torch
        import torch.nn as nn
        from torch.nn.functional import sigmoid, relu
        from scipy.cluster.hierarchy import dendrogram, linkage
```

Let's first load in the names of all the items, attributes, and relations into Python lists.

```
In [2]: with open('data/sem_items.txt','r') as fid:
        names_items = np.array([l.strip() for l in fid.readlines()])
        with open('data/sem_relations.txt','r') as fid:
            names_relations = np.array([l.strip() for l in fid.readlines()])
        with open('data/sem_attributes.txt','r') as fid:
            names_attributes = np.array([l.strip() for l in fid.readlines()])

        nobj = len(names_items)
        nrel = len(names_relations)
        nattributes = len(names_attributes)
        print('List of items:')
        print(names_items)
        print("List of relations:")
        print(names_relations)
        print("List of attributes:")
        print(names_attributes)
```

List of items:

['Pine' 'Oak' 'Rose' 'Daisy' 'Robin' 'Canary' 'Sunfish' 'Salmon']

List of relations:

['ISA' 'Is' 'Can' 'Has']

List of attributes:

['Living thing' 'Plant' 'Animal' 'Tree' 'Flower' 'Bird' 'Fish' 'Pine'
'Oak' 'Rose' 'Daisy' 'Robin' 'Canary' 'Sunfish' 'Salmon' 'Pretty' 'Big'
'Living' 'Green' 'Red' 'Yellow' 'Grow' 'Move' 'Swim' 'Fly' 'Sing' 'Skin'
'Roots' 'Leaves' 'Bark' 'Branch' 'Petals' 'Wings' 'Feathers' 'Gills'
'Scales']

Next, let's load in the data matrix from a text file too. The matrix `D` has a row for each training pattern. It is split into a matrix of input patterns `input_pats` (item and relation) and their corresponding output patterns `output_pats` (attributes). The are `N` patterns total in the set.

For each input pattern, the first 8 elements indicate which item is being presented, and the next 4 indicate which relation is being queried. Each element of the output pattern corresponds to a different attribute. All patterns use 1-hot encoding.

```
In [3]: D = np.loadtxt('data/sem_data.txt')
input_pats = D[:, :nobj+nrel]
input_pats = torch.tensor(input_pats, dtype=torch.float)
output_pats = D[:, nobj+nrel:]
output_pats = torch.tensor(output_pats, dtype=torch.float)
N = input_pats.shape[0] # number of training patterns
input_v = input_pats[0,:].numpy().astype('bool')
output_v = output_pats[0,:].numpy().astype('bool')
print('Example input pattern:')
print(input_v.astype('int'))
print('Example output pattern:')
print(output_v.astype('int'))
print("")
print("Which encodes...")
print('Item ', end='')
print(names_items[input_v[:8]])
print('Relation ', end='')
print(names_relations[input_v[8:]])
print('Attributes ', end='')
print(names_attributes[output_v])
```

Example input pattern:

[1 0 0 0 0 0 0 0 1 0 0 0]

Example output pattern:

[1 1 0 1 0 0 0 1 0]

Which encodes...

Item ['Pine']

Relation ['ISA']

Attributes ['Living thing' 'Plant' 'Tree' 'Pine']

Problem 1 (20 points)

Your assignment is to create the neural network architecture shown in the figure above. Fill in the missing pieces of the "Net" class in the code below. For an example, refer to the PyTorch tutorial on ["Neural Networks"](#).

Use the ReLu activation function ("relu") for the Representation and Hidden Layers, with a Logistic/Sigmoid activation function for the Attribute Layer ("sigmoid").

You will need PyTorch's "nn.Linear" function for constructing the layers, and the "relu" and "sigmoid" activation functions.

```
In [4]: class Net(nn.Module):
        def __init__(self, rep_size, hidden_size):
```

```

    super(Net, self).__init__()
    # Input
    # rep_size : number of hidden units in "Representation Layer"
    # hidden_size : number of hidden units in "Hidden Layer"
    #
    # TODO : YOUR CODE GOES HERE
    self.i2rep = nn.Linear(nobj, rep_size) # item to representation
    self.repRel2hidden = nn.Linear(rep_size + nrel, hidden_size) # representation + relation to hidden
    self.hidden2out = nn.Linear(hidden_size, nattributes) # hidden to output

def forward(self, x):
    # Defines forward pass for the network on input patterns x
    #
    # Input can take these two forms:
    #
    # x: [nobj+nrel 1D Tensor], which is a single input pattern as a 1D vector
    #     (containing both object and relation 1-hot identifier) (batch size = 1)
    # OR
    # x : [B x (nobj+nrel) Tensor], which is a batch of B input patterns
    #
    # Output
    # output [B x nattribute Tensor], which is the output pattern for the batch
    # hidden [B x hidden_size Tensor], which are activations in the Hidden Layer
    # rep [B x rep_size Tensor], which are the activations in the Representation Layer
    x = x.view(-1, nobj+nrel) # reshape as size [B x (nobj+nrel) Tensor]
    x_pat_item = x[:, :nobj] # input to Item Layer [B x nobj Tensor]
    x_pat_rel = x[:, nobj:] # input to Relation Layer [B x nrel Tensor]
    # TODO : YOUR CODE GOES HERE
    # ----
    rep = relu(self.i2rep(x_pat_item))
    hidden = relu(self.repRel2hidden(torch.cat((rep, x_pat_rel), dim=1)))
    output = sigmoid(self.hidden2out(hidden))
    # -----
    return output, hidden, rep

```

We provide a completed function `train` for stochastic gradient descent. The network makes online (rather than batch) updates, adjusting its weights after the presentation of each input pattern.

```

In [5]: def train(mynet, epoch_count, nepochs_additional=5000):
    # Input
    # mynet : Net class object
    # epoch_count : (scalar) how many epochs have been completed so far
    # nepochs_additional : (scalar) how many more epochs we want to run
    mynet.train()
    for e in range(nepochs_additional): # for each epoch
        error_epoch = 0.
        perm = np.random.permutation(N)
        for p in perm: # iterate through input patterns in random order
            mynet.zero_grad() # reset gradient
            output, hidden, rep = mynet(input_pats[p,:]) # forward pass
            target = output_pats[p,:]
            loss = criterion(output, target) # compute loss
            loss.backward() # compute gradient
            optimizer.step() # update network parameters

```

```

        error_epoch += loss.item()
        error_epoch = error_epoch / float(N)
        if e % 50 == 0:
            print('epoch ' + str(epoch_count+e) + ' loss ' + str(round(error
return epoch_count + nepochs_additional

```

We provide some useful functions for extracting the activation pattern on the Representation Layer for each possible item. We provide two functions `plot_rep` and `plot_dendo` for visualizing these activation patterns.

```

In [6]: def get_rep(net):
        # Extract the hidden activations on the Representation Layer for each it
        #
        # Input
        # net : Net class object
        #
        # Output
        # rep : [nitem x rep_size numpy array], where each row is an item
        input_clean = torch.zeros(nobj,nobj+nrel)
        for idx,name in enumerate(names_items):
            input_clean[idx,idx] = 1. # 1-hot encoding of each object (while Rel
        output, hidden, rep = mynet(input_clean)
        return rep.detach().numpy()

def plot_rep(rep1,rep2,rep3,names):
    # Compares Representation Layer activations of Items at three different
    # using bar graphs
    #
    # Each rep1, rep2, rep3 is a [nitem x rep_size numpy array]
    # names : [nitem list] of item names
    #
    nepochs_list = [nepochs_phase1,nepochs_phase2,nepochs_phase3]
    nrows = nobj
    R = np.dstack((rep1,rep2,rep3))
    mx = R.max()
    mn = R.min()
    depth = R.shape[2]
    count = 1
    plt.figure(1,figsize=(4.2,8.4))
    for i in range(nrows):
        for d in range(R.shape[2]):
            plt.subplot(nrows, depth, count)
            rep = R[i,:,d]
            plt.bar(range(rep.size),rep)
            plt.ylim([mn,mx])
            plt.xticks([])
            plt.yticks([])
            if d==0:
                plt.ylabel(names[i])
            if i==0:
                plt.title("epoch " + str(nepochs_list[d]))
            count += 1
    plt.show()

def plot_dendo(rep1,rep2,rep3,names):

```

```

# Compares Representation Layer activations of Items at three different
# using hierarchical clustering
#
# Each rep1, rep2, rep3 is a [nitem x rep_size numpy array]
# names : [nitem list] of item names
#
nepochs_list = [nepochs_phase1,nepochs_phase2,nepochs_phase3]
linked1 = linkage(rep1,'single')
linked2 = linkage(rep2,'single')
linked3 = linkage(rep3,'single')
mx = np.dstack((linked1[:,2],linked2[:,2],linked3[:,2])).max()+0.1
plt.figure(2,figsize=(7,12))
plt.subplot(3,1,1)
dendrogram(linked1, labels=names, color_threshold=0)
plt.ylim([0,mx])
plt.title('Hierarchical clustering; ' + "epoch " + str(nepochs_list[0]))
plt.ylabel('Euclidean distance')
plt.subplot(3,1,2)
plt.title("epoch " + str(nepochs_list[1]))
dendrogram(linked2, labels=names, color_threshold=0)
plt.ylim([0,mx])
plt.subplot(3,1,3)
plt.title("epoch " + str(nepochs_list[2]))
dendrogram(linked3, labels=names, color_threshold=0)
plt.ylim([0,mx])
plt.show()

```

The next script initializes the neural network and trains it for 2500 epochs total. It trains in three stages, and the item representations (on the Representation Layer) are extracted after 500 epochs, 1000 epochs, and then at the end of training (2500 epochs).

```

In [7]: learning_rate = 0.1
criterion = nn.MSELoss() # mean squared error loss function
mynet = Net(rep_size=8,hidden_size=15)
optimizer = torch.optim.SGD(mynet.parameters(), lr=learning_rate) # stochastic

nepochs_phase1 = 500
nepochs_phase2 = 1000
nepochs_phase3 = 2500
epoch_count = 0
epoch_count = train(mynet,epoch_count,nepochs_additional=nepochs_phase1)
rep1 = get_rep(mynet)
epoch_count = train(mynet,epoch_count,nepochs_additional=nepochs_phase2-nepochs_phase1)
rep2 = get_rep(mynet)
epoch_count = train(mynet,epoch_count,nepochs_additional=nepochs_phase3-nepochs_phase2)
rep3 = get_rep(mynet)

```

```

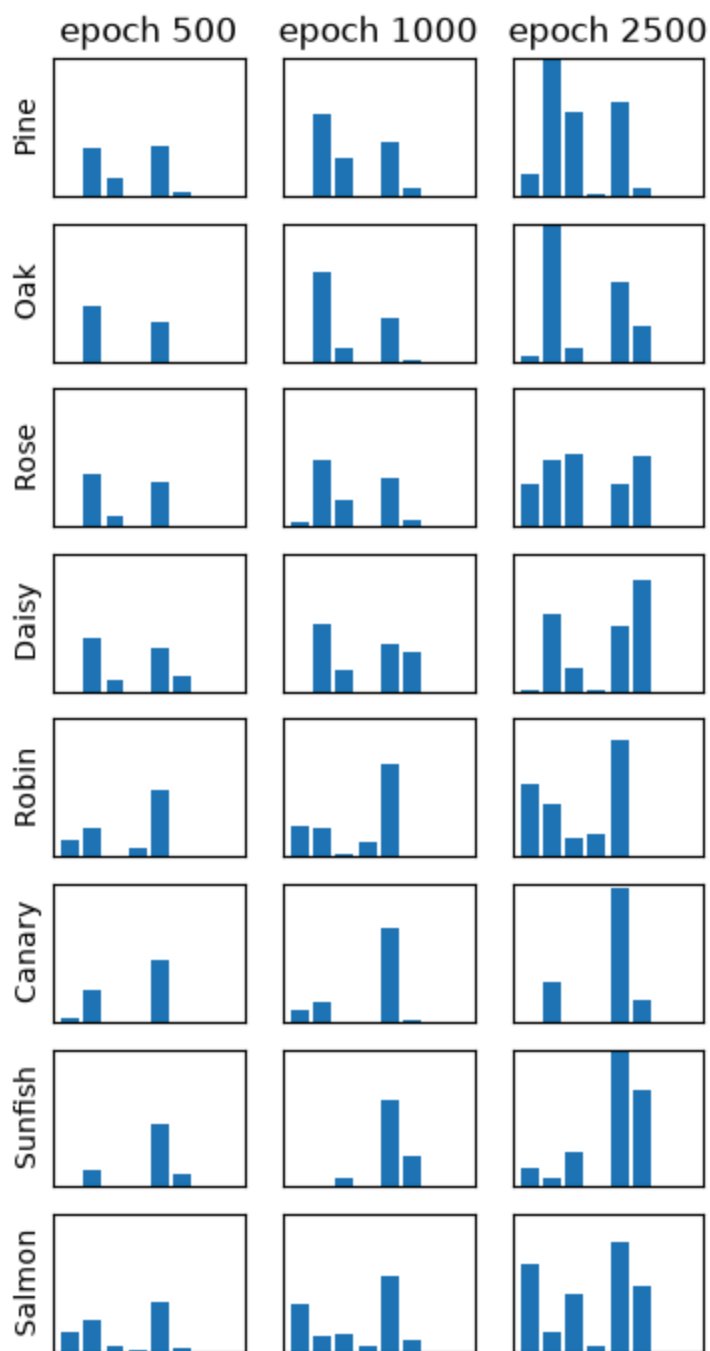
/Users/bmart231/CMC2026/.venv/lib/python3.13/site-packages/torch/nn/modules/loss.py:630: UserWarning: Using a target size (torch.Size([36])) that is different to the input size (torch.Size([1, 36])). This will likely lead to incorrect results due to broadcasting. Please ensure they have the same size.
  return F.mse_loss(input, target, reduction=self.reduction)

```

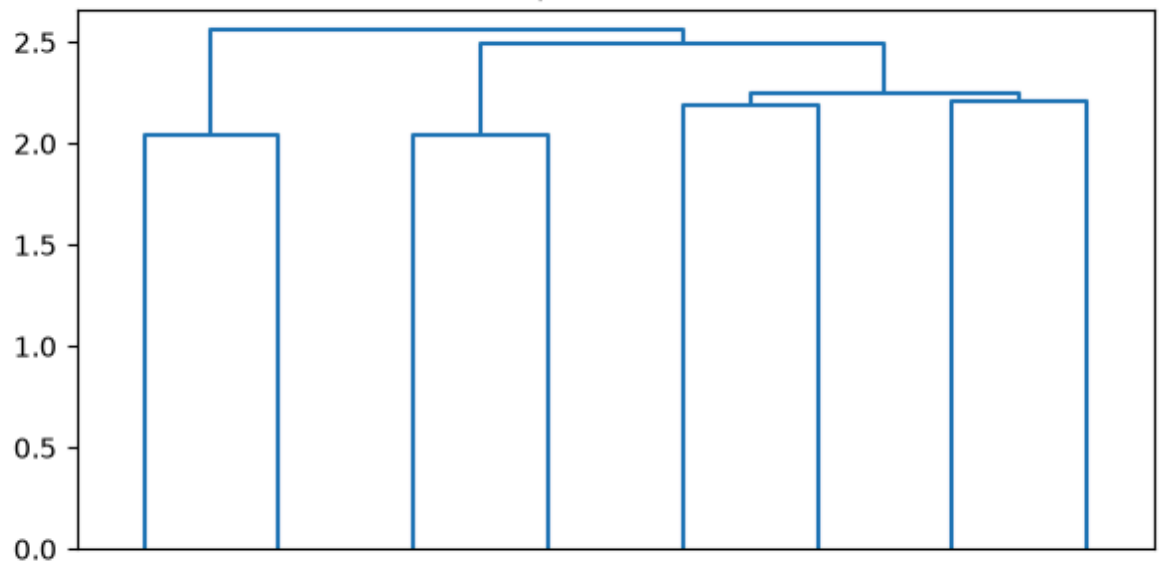
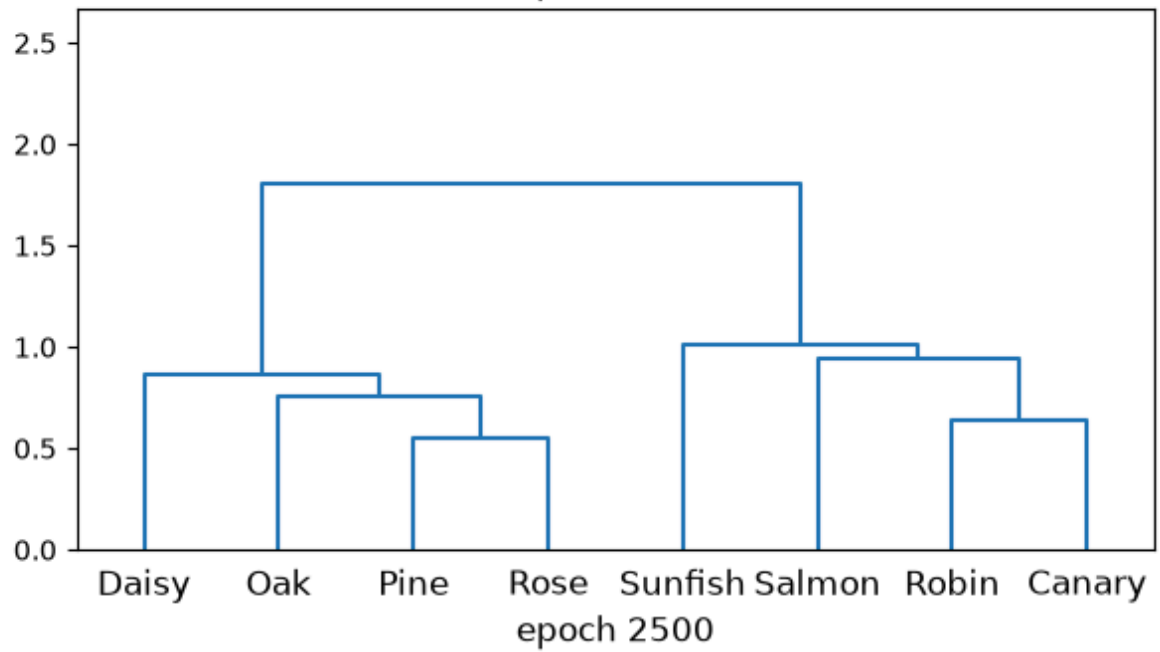
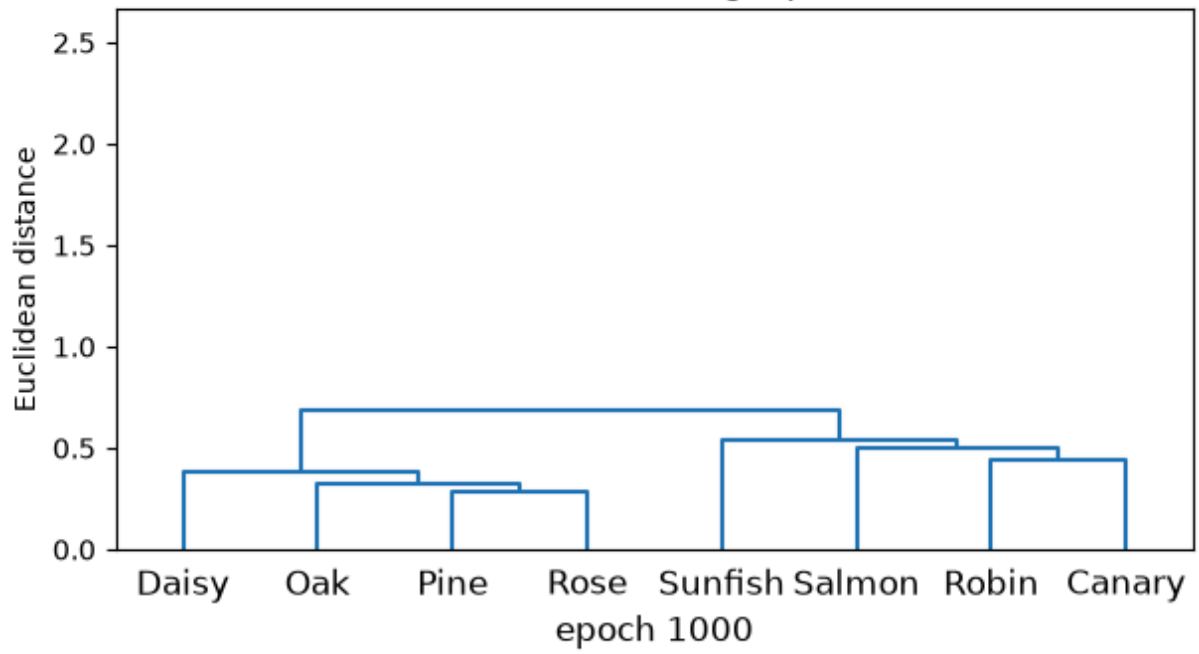
```
epoch 0 loss 0.252
epoch 50 loss 0.069
epoch 100 loss 0.066
epoch 150 loss 0.06
epoch 200 loss 0.054
epoch 250 loss 0.05
epoch 300 loss 0.048
epoch 350 loss 0.046
epoch 400 loss 0.044
epoch 450 loss 0.043
epoch 500 loss 0.041
epoch 550 loss 0.038
epoch 600 loss 0.034
epoch 650 loss 0.031
epoch 700 loss 0.028
epoch 750 loss 0.027
epoch 800 loss 0.026
epoch 850 loss 0.025
epoch 900 loss 0.024
epoch 950 loss 0.023
epoch 1000 loss 0.023
epoch 1050 loss 0.021
epoch 1100 loss 0.019
epoch 1150 loss 0.017
epoch 1200 loss 0.014
epoch 1250 loss 0.012
epoch 1300 loss 0.01
epoch 1350 loss 0.009
epoch 1400 loss 0.007
epoch 1450 loss 0.007
epoch 1500 loss 0.006
epoch 1550 loss 0.005
epoch 1600 loss 0.005
epoch 1650 loss 0.004
epoch 1700 loss 0.004
epoch 1750 loss 0.004
epoch 1800 loss 0.003
epoch 1850 loss 0.003
epoch 1900 loss 0.003
epoch 1950 loss 0.003
epoch 2000 loss 0.003
epoch 2050 loss 0.003
epoch 2100 loss 0.003
epoch 2150 loss 0.002
epoch 2200 loss 0.002
epoch 2250 loss 0.002
epoch 2300 loss 0.002
epoch 2350 loss 0.002
epoch 2400 loss 0.002
epoch 2450 loss 0.002
```

Finally, let's visualize the Representation Layer at the different stages of learning.

```
In [8]: plot_rep(rep1, rep2, rep3, names_items)
        plot_dendo(rep1, rep2, rep3, names_items)
```



Hierarchical clustering; epoch 500



Problem 2 (5 points)

Based on your plots, write a short analysis (4-5 sentences) of how the internal representations of the network develop over the course of learning. How does learning progress? Does the network start by differentiating certain classes of patterns from each other, and then differentiate others in later stages?

Hint: You can refer to your lecture slides and notes for the R&M model for help with your analysis. Your network should broadly replicate their findings, but since the training patterns and activation function aren't identical, don't expect the exact same results.

Answer: Early in training (epoch 500), the network already splits plants from animals, but all distances are small (under 0.7), so this is only a rough distinction and items within each group look very similar. By epoch 1000, the plant vs. animal split is much clearer (around 1.8), and within the animals the birds (robin and canary) have started to pair up, but finer plant groups have not formed yet. By epoch 2500, the distances are much larger (about 2.0 or more) and the network has learned the specific classes: trees (pine and oak), flowers (rose and daisy), fish, and birds each form their own pair. Overall, learning moves from broad to specific, which matches Rogers and McClelland's idea of progressive differentiation, because attributes shared by many items (like those shared by all animals) give large, consistent error signals early, while item-specific attributes are learned later. One difference from R&M is that in the final dendrogram the flowers join the animals before joining the trees, likely because of the ReLU activations and the random starting weights.

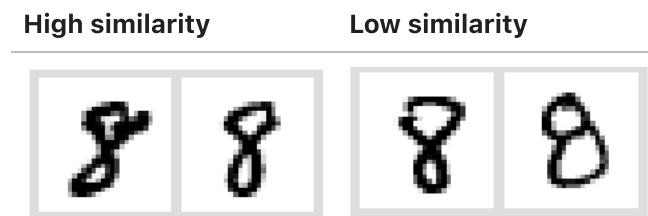
Homework - Neural networks - Part D (20 points)

Predicting similarity ratings with a deep convolutional network

by Brenden Lake

Summary: We will examine whether deep convolutional neural networks (convnets) trained for image classification can predict human similarity ratings for handwritten digits. The images below are all clear examples of the digit '8'. Nonetheless, the left two '8's look more "similar" to me than the right two '8's. Can a neural network trained for classification help to explain judgments such as this one?

In this notebook, we will use a pre-trained convnet for digit recognition to predict similarity judgments. We will then test the network's predictions by collecting similarity ratings from a couple of friends. *This assignment requires collecting a small amount of behavioral data, so grab a friend and don't wait to the last minute to get started!*



Background

The goal of this assignment is to give you some hands-on experience with a powerful toolkit for computer vision (deep convnets), which has also recently been applied for studying human perception and categorization.

In 2012, Krizhevsky, Sutskever, and Hinton trained a deep convnet (now called 'AlexNet') for object recognition and achieved very impressive results, reducing the number of errors on ImageNet by almost half compared to the next best approach. This paper ignited the recent deep learning revolution in computer vision, although convnets were being used for this purpose long before (LeCun et al. (1989) was the first to train deep convnets for digit recognition).

Recently, the success of convnets has led researchers in cognitive science and neuroscience to explore their potential applications for understanding human visual perception and categorization. Here are some examples: Yamins et al. (2014) showed

that deep convnets can predict neural response in high-level visual areas; Lake, Zaremba, Fergus, and Gureckis (2015) showed that deep convnets can explain human typicality ratings for some classes of natural images; Peterson, Abbott, and Griffiths (2016) explored convnets for predicting similarity ratings between images.

In this assignment, like Peterson et al., we will explore convnets for predicting human similarity judgments between images. We use a relatively small-scale network trained for digit recognition, but the same principles can be used to study much more complex deep convnets trained for object recognition. Although "similarity" can be a tricky construct in cognitive science (Medin, Goldstone, & Gentner, 1993), similarity judgments are still a useful tool in a cognitive scientist's toolbox for understanding representation.

References:

- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In Advances in Neural Information Processing Systems.
- Lake, B. M., Zaremba, W., Fergus, R., & Gureckis, T. M. (2015). Deep Neural Networks Predict Category Typicality Ratings for Images. In Proceedings of the Cognitive Science Society.
- LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. Neural Computation, 1(4), 541-551.
- Medin, D. L., Goldstone, R. L., & Gentner, D. (1993). Respects for similarity. Psychological Review, 100(2), 254.
- Peterson, J. C., Abbott, J. T., & Griffiths, T. L. (2016). Adapting deep network features to capture psychological representations. arXiv preprint arXiv:1608.02164.
- Yamins, D. L., Hong, H., Cadieu, C. F., Solomon, E. A., Seibert, D., & DiCarlo, J. J. (2014). Performance-optimized hierarchical models predict neural responses in higher visual cortex. Proceedings of the National Academy of Sciences, 111(23), 8619-8624.

Downloading MNIST

MNIST is one of the most famous data sets in machine learning. It is a digit recognition task, where the goal is classify images of handwritten digits with the right label, e.g., either '0','1','2', ... '9'. The training set consists of 60,000 images (6,000 per digit), and the test set has 10,000 additional images.

Execute the code below to load some libraries and the MNIST "test set." We will not need the training set, since we will use a pre-trained network for this assignment.

```
In [4]: from __future__ import print_function
        %matplotlib inline
```

```

import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms, utils
from torch.autograd import Variable
import os
import numpy as np
from scipy.spatial.distance import cosine
import random

print('Loading MNIST')
test_loader = torch.utils.data.DataLoader(datasets.MNIST('~/.shared',
                                                         train=False,
                                                         download=True,
                                                         transform=transforms.Compose([
                                                             transforms.ToTensor(),
                                                             transforms.Normalize([0.1307], [0.3081])
                                                         ]), batch_size=1000, shuffle=True))

print('MNIST has been loaded.')

```

Loading MNIST
MNIST has been loaded.

Convnet architecture for digit recognition

Here is the `Net` class defining the network we are using for digit recognition.

First convolutional layer:

This layer takes an image (28x28 for MNIST) and applies a bank of learnable 5x5 filters to produce 10 new images (also known as channels or feature maps). Each feature map is passed through a non-linearity (a rectified linear unit or "ReLU"). Last, there is a max pooling operation that reduces the size of each feature map by half.

Second convolutional layer:

This layer takes the feature maps from the previous layer, applies a bank of learnable 5x5 filters, and produces 20 new feature maps. Again, ReLU's are applied as well as max pooling.

Fully-connected layer:

Next is a standard fully-connected layer of 50 units. At this stage, the entire image is summarized with a vector of size 50. ReLU's are used again.

Output layer: The output layer has 10 units with one to represent each of the 10 digits. It uses a softmax activation function, to ensure the network's predictions are a valid probability distribution of the 10 possibilities.

Execute the code to define the `Net` class.

```
In [11]: class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 10, kernel_size=5) # 10 channels in first
        self.conv2 = nn.Conv2d(10, 20, kernel_size=5) # 20 channels in second
        self.fc1 = nn.Linear(320, 50) # 50 hidden units in first fully-connected
        self.fc2 = nn.Linear(50, 10) # 10 output units

    def forward(self, x):

        # first convolutional layer
        h_conv1 = self.conv1(x)
        h_conv1 = F.relu(h_conv1)
        h_conv1_pool = F.max_pool2d(h_conv1, 2)

        # second convolutional layer
        h_conv2 = self.conv2(h_conv1_pool)
        h_conv2 = F.relu(h_conv2)
        h_conv2_pool = F.max_pool2d(h_conv2, 2)

        # fully-connected layer
        h_fc1 = h_conv2_pool.view(-1, 320)
        h_fc1 = self.fc1(h_fc1)
        h_fc1 = F.relu(h_fc1)

        # classifier output
        output = self.fc2(h_fc1)
        output = F.log_softmax(output, dim=1)
        return output, h_fc1, h_conv2, h_conv1
```

Evaluating classification performance of the network

Here we will execute our first bit of non-trivial code. To save time, we already trained a MNIST network for you. The code below loads the pre-trained model. If you are curious, it only takes a couple minutes to train a MNIST model from scratch on a standard laptop, and we used a version of the PyTorch code from [here](#) to train this network.

After loading the network, the function `test_all` iterates through the whole test set and computes the network's predicted class for each image. It outputs the percent correct.

The function `test_viz` picks a few images from the test set at random and displays them along with the network's predicted class labels.

Execute the code below. If everything is working right, **you should get test performance of 98.71%** correct which isn't state-of-the-art, but is reasonable. You can also see some of the network's specific predictions on some test images, which should also look good. You can re-run to see a few different sets of test images.

```

In [6]: # Evaluate classification accuracy on the entire MNIST test set
def test_all():
    correct = 0
    for data, target in test_loader:
        # run the data through the network
        output, h_fc1, h_conv2, h_conv1 = model(data)
        # compare prediction to ground truth
        pred = torch.max(output, dim=1)[1] # get the index of the max log-probability
        correct += torch.eq(pred, target.view_as(pred)).cpu().sum().item()
    perc_correct = 100. * correct / len(test_loader.dataset)
    return perc_correct

# Show the network's predicted class for an arbitrary set of `nshow` images
def test_viz(nshow=10):

    # grab a random subset of the data
    testiter = iter(test_loader)
    images, target = next(testiter)
    perm = np.random.permutation(images.shape[0])
    data = images[perm[:nshow]]

    # get predictions from the network
    output, h_fc1, h_conv2, h_conv1 = model(data)
    pred = torch.max(output, dim=1)[1]
    pred = pred.numpy().flatten()

    # plot predictions along with the images
    for i in range(nshow):
        ax = plt.subplot(1, nshow, i+1)
        imshow(utils.make_grid(data[i]))
        plt.title(str(pred[i]))

# Display an image from the MNIST data set
def imshow(img):
    img = 1 - (img * 0.3081 + 0.1307) # invert image pre-processing
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.axis('off')

checkpoint = torch.load('models/convnet_mnist_pretrained.pt')
model = Net()
model.load_state_dict(checkpoint['state_dict'])
model.eval()
print('Convnet has been loaded successfully.')
print('Running through the test set...')
test_acc = test_all()
print(' Accuracy on the test set is %.2f percent correct!' % test_acc)
print("")
print("Here are some predictions from the network.")
print("The images are shown below their predicted class labels.")
test_viz()

```

Convnet has been loaded successfully.
Running through the test set...
Accuracy on the test set is 98.71 percent correct!

Here are some predictions from the network.
The images are shown below their predicted class labels.

9	6	6	0	5	0	2	5	5	0
9	6	6	0	5	0	2	5	5	0

Selecting digits for similarity analysis

Here are two more useful functions: `get_random_subset` and `plot_image_pairs`.

As mentioned, we are looking at within-class similarity comparisons (e.g., comparing two different images of the digit '8'). The function `get_random_subset` generates a set of random images pairs from a particular image class `digit_select` from the MNIST test set. The number of random pairs can be set by `npairs`.

The other function is `plot_image_pairs` which visualizes each of the random image pairs. The pairs are input as two list of image tensors, where images with the same index are paired.

```
In [7]: def get_random_subset(digit_select, npairs=20):
        # digit_select: which digit do we want to get images for?
        testiter = iter(test_loader)
        images, target = next(testiter)
        indices = np.flatnonzero(target.numpy() == digit_select)
        np.random.shuffle(indices)
        indx1 = torch.tensor(indices[:npairs], dtype=torch.long)
        indx2 = torch.tensor(indices[npairs:npairs*2], dtype=torch.long)
        images1 = images[indx1]
        images2 = images[indx2]
        plt.figure(1, figsize=(4, 40))
        plot_image_pairs(images1, images2)
        return images1, images2

def plot_image_pairs(images1, images2, scores_net=[], scores_people=[]):
    # images1 : list of images (each image is a tensor)
    # images2 : list of images (each image is a tensor)
    # scores_net (optional) : network similarity score for each pair
    # scores_people (optional) : human similarity score for each pair
    npairs = images1.size()[0]
    assert images2.size()[0] == npairs
    for i in range(npairs):
        ax = plt.subplot(npairs, 1, i+1)
        imshow(utils.make_grid([images1[i], images2[i]]))
        mytitle = ''
        if len(scores_net)>0:
            mytitle += 'net %.2f, ' % scores_net[i]
```

```
if len(scores_people)>0:
    mytitle += 'human %.2f' % scores_people[i]
if mytitle:
    plt.title(mytitle)
```

Here is code for sampling 20 random pairings of images of the digit '8' (or whichever digit you set `digit_select` to). Run to sample the pairings and then visualize them (you may have to scroll).

To save or print out the image pairs, which will be helpful later in the assignment, you can right click and save the output as an image.

```
In [8]: images1_digit8, images2_digit8 = get_random_subset(digit_select=8, npairs = 20)
```









Computing similarity judgments with the network

To get predictions out of the network, we take the hidden representation at a particular layer as a representation of an image. The hidden representation computes high-level features of the images, which we will use to compare the images. To compute the similarity between two images, we get the two hidden vectors and compare them with the [cosine similarity](#) metric. Cosine similarity measures the angle between two vectors, which can be an effective way of measuring similarity between patterns of activation on the hidden layer.

The function `get_sim_judgments` takes two lists of images and computes the similarity between each pair. Using the `layer` parameter, you can choose between the fully connected layer (`fc`), first convolutional layer (`conv1`), or second convolutional layer (`conv2`).

The function `normalize` rescales a vector to have a minimum value of 0 and maximum value of 1.

```
In [9]: # re-scale a vector to have a minimum value of 0, maximum of 1
def normalize(v):
    # v : numpy vector
    v = v - v.min()
    v = v / v.max()
    return v

# Compute convnet similarity for each pair of images
def get_sim_judgments(images1, images2, layer='fc'):
    # images1 : list of N images (each image is a tensor)
    # images2 : list of N images (each image is a tensor)
    # layer : which layer do we want to use? fully connected ('fc'),
    #         first convolutional ('conv1'), or second convolutional ('conv2')
    #
    # Return
    # v_sim : N-dimensional vector
    N = images1.size()[0] # number of pairs
    assert N == images2.size()[0]
    output_1, h_fc1_1, h_conv2_1, h_conv1_1 = model(images1)
```

```

output_2, h_fc1_2, h_conv2_2, h_conv1_2 = model(images2)

# grab the tensors from the appropriate layer
if layer=='fc':
    T1 = h_fc1_1
    T2 = h_fc1_2
elif layer=='conv1':
    T1 = h_conv1_1
    T2 = h_conv1_2
elif layer=='conv2':
    T1 = h_conv2_1
    T2 = h_conv2_2
else:
    raise Exception('Layer parameter has unrecognized value')

# flatten the tensors for each image
T1 = T1.detach().numpy().reshape(N,-1)
T2 = T2.detach().numpy().reshape(N,-1)

v_sim = np.zeros(N)
for i in range(N): # for each pair
    v1 = T1[i,:]
    v2 = T2[i,:]
    v_sim[i] = 1-cosine(v1,v2) # using cosine distance
return v_sim

# Get similarity bsaed on the fully connected layer at the end
v_sim_net_digit8 = get_sim_judgments(images1_digit8,images2_digit8,'fc')
print("Similarity ratings from the neural network the for digit 8:")
print(v_sim_net_digit8)

```

```

Similarity ratings from the neural network the for digit 8:
[0.88444179 0.82469904 0.58894801 0.74628973 0.84821761 0.81817889
 0.55951083 0.79489172 0.50642914 0.93558472 0.91972327 0.84651893
 0.72101593 0.75043005 0.74635845 0.84543294 0.8504377 0.84972501
 0.75429296 0.70905221]

```

Problem 1 (10 points)

Go through the following steps for the digit '8':

1. *Get judgments from the network.* Run all of the code above to sample a list of 20 pairs of images for that digit. For each pair, get a measure of the network's similarity between the images as measured on the fully-connected `fc` layer. (You may have already done all of this.)
2. *Get judgments from people.* Ask two people to rate the same image pairs on a 1 (least similar) to 10 (most similar) scale, using the question "How similar do these two images look to you?" (You can right-click and save the jupyter notebook figures as images). Ask your participant (in a socially distanced way) to

rate each pair. Enter the ratings in the arrays `ratings\_human\_1\_digit8` and `ratings\_human\_2\_digit8` below. One participant can be yourself, but the other should be someone else.

3. **Compare.** Run the code below to compute the correlation coefficient and scatter plot.

```
In [13]: # YOUR PARTICIPANT RATINGS GO HERE
ratings_human_1_digit8 = np.array([10,5,6,8,8,2,4,9,3,9,8,6,5,5,3,9,8,3,8,8])
ratings_human_2_digit8 = np.array([9,3,5,8,5,3,4,8,3,8,8,5,5,8,4,4,4,6,6,3])
# Analysis code
ratings_human_mean_digit8 = (ratings_human_1_digit8 + ratings_human_2_digit8)

v_sim_net_norm = normalize(v_sim_net_digit8)
v_sim_human_norm = normalize(ratings_human_mean_digit8)

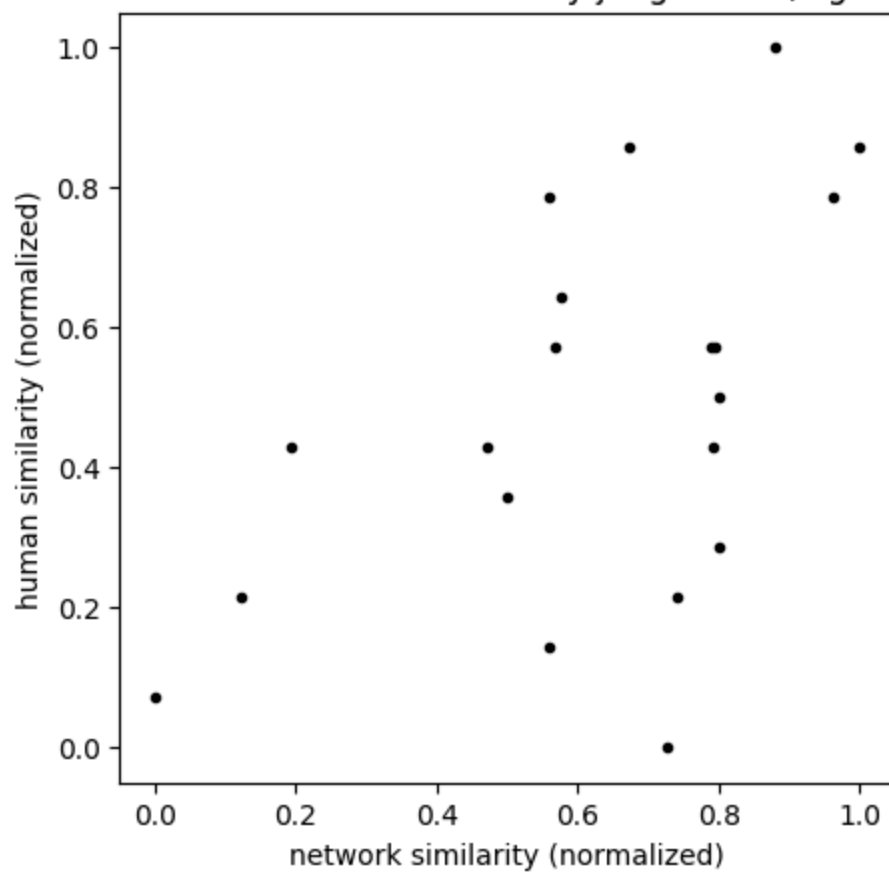
print("Correlation between net and human similarity ratings: r ="),
print(round(np.corrcoef(v_sim_net_norm,v_sim_human_norm)[0][1],3))

# Scatter plot
plt.figure(1,figsize=(5,5))
plt.plot(v_sim_net_norm,v_sim_human_norm,'k.')
plt.xlabel('network similarity (normalized)')
plt.ylabel('human similarity (normalized)')
plt.title('Convnet vs. human similarity judgments (digit 8)')
plt.show()

# pairs with similarity ratings
plt.figure(2,figsize=(4,40))
plot_image_pairs(images1_digit8,images2_digit8,v_sim_net_norm,v_sim_human_norm)
plt.show()
```

Correlation between net and human similarity ratings: r =
0.508

Convnet vs. human similarity judgments (digit 8)



net 0.88, human 1.00



net 0.74, human 0.21



net 0.19, human 0.43



net 0.56, human 0.79



net 0.80, human 0.57



net 0.73, human 0.00



net 0.12, human 0.21



net 0.67, human 0.86



net 0.00, human 0.07



net 1.00, human 0.86



net 0.96, human 0.79



net 0.79, human 0.43



net 0.50, human 0.36



net 0.57, human 0.57



net 0.56, human 0.14



net 0.79, human 0.57



net 0.80, human 0.50



net 0.80, human 0.29



net 0.58, human 0.64





Problem 2 (10 points)

How well does the convolutional neural network do at predicting the similarity ratings? What is it capturing? What is it not capturing? (Keep your answer brief. About 5 sentences is good.)

Don't worry if your fit isn't very good! A convnet is very far from a perfect account of human similarity ratings. I had reasonable success with the '8' digit but I wouldn't expect a correlation higher than $r=0.6$. This assignment is not graded on how good the fit is.

Answer: The network did a moderately good job of predicting the similarity ratings, with a correlation of $r = 0.51$, which is about as high as the two people agreed with each other ($r = 0.54$). It seems to capture the overall shape of the digits, like slant, proportions, and loop size as pairs that looked alike to people (such as pairs 1 and 10) also got high network scores, and pairs that differed in several ways (such as pairs 7 and 9) got low scores from both. However, it misses differences in stroke thickness, smudged or filled-in strokes, and messy handwriting. For example, in pairs 6, 15, and 18 people saw the two 8s as very different because one was a dark blob or had a broken loop, but the network still rated them as fairly similar. This makes sense because the network was only trained to classify digits, so it learned to ignore differences that don't change the label "8," while people notice those differences when judging similarity.

Homework - Neural networks - Part E (50 points)

Discovering lexical classes from simple sentences

by Brenden Lake

In this assignment, you will follow in Elman's (1990) footsteps by coding and training a Simple Recurrent Network (SRN) on a set of simple sentences.

- **Before training**, the SRN can process sequences but otherwise knows nothing about language. Initially, it represents each word as an arbitrary continuous vector (input embedding) without knowledge of their roles or how they relate to each other.
- **During training**, the SRN aims to predict the next word in a sentence given the previous words. The optimizer takes a step after each sentence.
- **After training**, you will analyze the SRN's internal representations (input embeddings) for evidence that it has discovered something about lexical classes (e.g., nouns and verbs).

Make sure you submit a fully executed (top to bottom) version of this notebook! This is needed for the autograder.

Reference (available for download on Canvas):

Elman, J. L. (1990). Finding Structure in Time. Cognitive Science, 14:179–211.

```
In [1]: # Let's start with some packages we need
from __future__ import print_function
import torch
import torch.nn as nn
import numpy as np
import matplotlib
%matplotlib inline
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
```

Elman's set of simple sentences

The training set consists of 10,000 sentences each with 2 or 3 words. Elman generated each sentence as follows:

1. Choose one of 16 templates specifying a sequence of lexical classes (see below)

- Each lexical class is replaced by a word sampled from that class. (see below, only a subset of words shown).

The vocabulary contained 29 words. For example, the template **NOUN-AGRESS VERB-EAT NOUN-FOOD** can lead to the sentence **dragon eat cookie** along with other possibilities. We generated 10,000 sentences using our best guess of Elman's procedure (the full set of lexical classes isn't listed). You can see these sentences in the external text file `data/elman_sentences.txt`

TABLE 3
Categories of Lexical Items Used in Sentence Simulation

Category	Examples
NOUN-HUM	man, woman
NOUN-ANIM	cat, mouse
NOUN-INANIM	book, rock
NOUN-AGRESS	dragon, monster
NOUN-FRAG	glass, plate
NOUN-FOOD	cookie, break
VERB-INTRAN	think, sleep
VERB-TRAN	see, chase
VERB-AGPAT	move, break
VERB-PERCEPT	smell, see
VERB-DESTROY	break, smash
VERB-EAT	eat

TABLE 4
Templates for Sentence Generator

WORD 1	WORD 2	WORD 3
NOUN-HUM	VERB-EAT	NOUN-FOOD
NOUN-HUM	VERB-PERCEPT	NOUN-INANIM
NOUN-HUM	VERB-DESTROY	NOUN-FRAG
NOUN-HUM	VERB-INTRAN	
NOUN-HUM	VERB-TRAN	NOUN-HUM
NOUN-HUM	VERB-AGPAT	NOUN-INANIM
NOUN-HUM	VERB-AGPAT	
NOUN-ANIM	VERB-EAT	NOUN-FOOD
NOUN-ANIM	VERB-TRAN	NOUN-ANIM
NOUN-ANIM	VERB-AGPAT	NOUN-INANIM
NOUN-ANIM	VERB-AGPAT	
NOUN-INANIM	VERB-AGPAT	
NOUN-AGRESS	VERB-DESTROY	NOUN-FRAG
NOUN-AGRESS	VERB-EAT	NOUN-HUM
NOUN-AGRESS	VERB-EAT	NOUN-ANIM
NOUN-AGRESS	VERB-EAT	NOUN-FOOD

Loading the data

The following code will load and process the set of simple sentences. As is common in neural networks for text and natural language processing, the sentence strings are first "tokenized" into a list of discrete elements (words in this case). Additionally, special tokens indicating the start-of-sentence `<SOS>` and end-of-sentence `<EOS>` are added at the beginning and end of the sentence, respectively. The SRN requires an input at every step and thus we use `<SOS>` as the first input when the SRN is predicting the first word as output. The SRN can self-terminate a sentence by producing `<EOS>` as an output. The dict `token_to_index` maps each token to a unique integer, which is the format that the SRN actually uses as input.

Running the code below will show you the dict `token_to_index` and how the first sentence `dragon break plate` is tokenized into integers. Make sure you understand how this works and how to map back and forth between the formats!

```
In [2]: def sentenceToTensor(tokens_list):
# Convert list of strings to tensor of token indices (integers)
#
# Input
# tokens_list : list of strings, e.g. ['<SOS>','lion','eat','man','<EOS>']
# Output
# 1D tensor of the same length (integers), e.g., tensor([ 2, 18, 13, 19])
assert(isinstance(tokens_list,list))
tokens_index = [token_to_index[token] for token in tokens_list]
return torch.tensor(tokens_index)

# load and process the set of simple sentences
with open('data/elman_sentences.txt','r') as fid:
    lines = fid.readlines()
sentences_str = [l.strip() for l in lines]
sentences_tokens = [s.split() for s in sentences_str]
sentences_tokens = [['<SOS>']+s+['<EOS>'] for s in sentences_tokens]
unique_tokens = sorted(set(sum(sentences_tokens,[])))
n_tokens = len(unique_tokens) # all words and special tokens
token_to_index = {t : i for i,t in enumerate(unique_tokens)}
index_to_token = {i : t for i,t in enumerate(unique_tokens)}
training_pats = [sentenceToTensor(s) for s in sentences_tokens] # python list
ntrain = len(training_pats)
print('mapping unique tokens to integers: %s \n' % token_to_index)
print('example sentence as string: %s \n' % ' '.join(sentences_tokens[0]))
print('example sentence as tensor: %s \n' % training_pats[0])
```

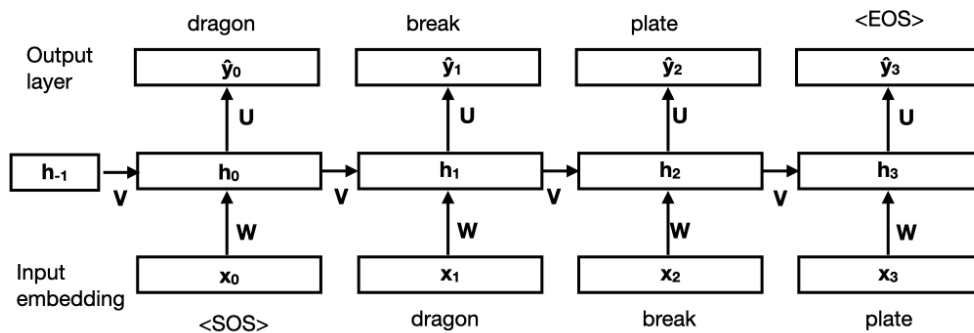
```
mapping unique tokens to integers: {'<EOS>': 0, '<SOS>': 1, 'book': 2, 'boy': 3, 'bread': 4, 'break': 5, 'car': 6, 'cat': 7, 'chase': 8, 'cookie': 9, 'dog': 10, 'dragon': 11, 'eat': 12, 'exist': 13, 'girl': 14, 'glass': 15, 'like': 16, 'lion': 17, 'man': 18, 'monster': 19, 'mouse': 20, 'move': 21, 'plate': 22, 'rock': 23, 'sandwich': 24, 'see': 25, 'sleep': 26, 'smash': 27, 'smell': 28, 'think': 29, 'woman': 30}
```

```
example sentence as string: <SOS> dragon break plate <EOS>
```

```
example sentence as tensor: tensor([ 1, 11,  5, 22,  0])
```

Simple Recurrent Network

The diagram below shows the unrolled SRN that you will develop here. As is always true for recurrent networks, notice the tied weights U , W , V , etc.



We will deviate from Elman's exact model in a few ways to make it more modern. Here is the specification we will use.

- **Input embedding.** In Elman's original model, each word was represented by a fixed one-hot input vector. Instead, here we will learn a continuous embedding vector (size `hidden_size=20`) to represent each input word. These vectors are learnable parameters. When a word is provided as input to the SRN, it is converted to the corresponding input embedding. This layer is setup for you already in the starter class, `self.embed = nn.Embedding(vocab_size, hidden_size)`
- **Hidden layer.** This layer has length `hidden_size` and uses the **logistic** activation function. The initial vector h_{-1} should be all zeros.
- **Output layer.** This layer has length `vocab_size` and uses the **softmax** activation function. Thus, the SRN will represent an explicit probability distribution over the next token w_j given the past tokens $w_1, \dots, w_{j-1}$, through the equation $P(w_j | w_1, \dots, w_{j-1})$
- **Loss.** The SRN will train to maximize the log-likelihood of the target output words, e.g., we use the negative log-likelihood loss `nn.NLLLoss`. If passed a tensor representing multiple target predictions, this loss takes the mean across predictions.
- **Optimizer.** We found reasonable results with the `AdamW` optimizer with weight decay of 0.04. Adam is like stochastic gradient descent but adapts the learning rate for each parameter based on the variance of the gradient. Weight decay encourages the parameters to be close to zero leading to more stable input embeddings.

- **Batching.** We suggest *no batching* for this simple code. Thus, the optimizer takes a step after each individual sentence. The `forward` method should process only one input word at a time. Batching produces much faster code and is recommended in practice, but it's not required here. If you want to rewrite the code to process multiple timesteps and sentences simultaneously, that's fine too.

Problem 1 (20 points)

Write code to complete the SRN class.

```
In [3]: class SRN(nn.Module):

    def __init__(self, vocab_size, hidden_size):
        # vocab_size : number of tokens in vocabulary including special tokens
        # hidden_size : dim of input embeddings and hidden layer
        super().__init__()
        self.vocab_size = vocab_size
        self.hidden_size = hidden_size
        self.embed = nn.Embedding(vocab_size, hidden_size)
        # TODO : YOUR CODE GOES HERE
        self.W = nn.Linear(hidden_size, hidden_size) # input embedding to hidden
        self.V = nn.Linear(hidden_size, hidden_size) # previous hidden to hidden
        self.U = nn.Linear(hidden_size, vocab_size) # hidden to output

    def forward(self, input_token_index, hidden_prev):
        # Input
        #   input_token_index: [integer] index of current input token
        #   hidden_prev: [length hidden_size 1D tensor] hidden state from previous step
        # Output
        #   output: [length vocab_size 1D tensor] log-probability of emitting next token
        #   hidden_curr : [length hidden_size 1D tensor] hidden state for current step
        input_embed = self.embed(input_token_index) # hidden_size 1D tensor
        # TODO : YOUR CODE GOES HERE
        hidden_curr = torch.sigmoid(self.W(input_embed) + self.V(hidden_prev))
        output = torch.log_softmax(self.U(hidden_curr), dim=-1) # log-probability
        return output, hidden_curr

    def initHidden(self):
        # Returns length hidden_size 1D tensor of zeros
        return torch.zeros(self.hidden_size)

    def get_embeddings(self):
        # Returns [vocab_size x hidden_size] numpy array of input embeddings
        return self.embed(torch.arange(self.vocab_size)).detach().numpy()
```

Problem 2 (20 points)

Write code to complete the `train` function and the main training loop. In the training loop, for each epoch, print out the mean loss over all training patterns. ****This should be printed as `mean\_loss\_for\_epoch`, which will be important for the autograder.**** An epoch should visit each sentence in random order, taking an optimizer step after each sentence.

Hint: In my implementation, after 10 epochs, I found that the mean loss to reach about 1.57. In other words, the SRN predicts the right word with roughly $e^{-1.57} = 0.208$ probability of getting it right. (Of course, perfect prediction is impossible in even this simple language).

```
In [4]: def train(seq_tensor, rnn):
    # Process a sentence and update the SRN weights. With <SOS> as the input
    # predict every subsequent word given the past words.
    # Return the mean loss across each symbol prediction.
    #
    # Input
    #   seq_tensor: [1D tensor] sentence as token indices
    #   rnn : instance of SRN class
    # Output
    #   loss : [scalar] average NLL loss across prediction steps

    # TODO : YOUR CODE GOES HERE
    optimizer.zero_grad()
    hidden = rnn.initHidden() # h_{-1} is all zeros
    outputs = []
    for t in range(len(seq_tensor)-1): # inputs are every token except the last
        output, hidden = rnn(seq_tensor[t], hidden)
        outputs.append(output)
    loss = criterion(torch.stack(outputs), seq_tensor[1:]) # targets are every token except the first
    loss.backward()
    optimizer.step()
    loss = loss.item()
    return loss
```

```
In [5]: # Main training loop
nepochs = 10 # number of passes through the entire training set
nhidden = 20 # number of hidden units in the SRN
rnn = SRN(n_tokens, nhidden)
optimizer = torch.optim.AdamW(rnn.parameters(), weight_decay=0.04) # w/ default
criterion = nn.NLLLoss()
# TODO : YOUR CODE GOES HERE
for epoch in range(nepochs):
    perm = np.random.permutation(ntrain) # visit sentences in random order
    total_loss = 0.
    for p in perm:
        total_loss += train(training_pats[p], rnn)
    mean_loss_for_epoch = total_loss / ntrain
    print('epoch %d, mean_loss_for_epoch %.4f' % (epoch, mean_loss_for_epoch))
print('The mean loss across all training sentences, for the final epoch, is')
```

epoch 0, mean_loss_for_epoch 1.7862
epoch 1, mean_loss_for_epoch 1.5873
epoch 2, mean_loss_for_epoch 1.5796
epoch 3, mean_loss_for_epoch 1.5776
epoch 4, mean_loss_for_epoch 1.5759
epoch 5, mean_loss_for_epoch 1.5759
epoch 6, mean_loss_for_epoch 1.5748
epoch 7, mean_loss_for_epoch 1.5740
epoch 8, mean_loss_for_epoch 1.5734
epoch 9, mean_loss_for_epoch 1.5729

The mean loss across all training sentences, for the final epoch, is 1.5729

Analyze the SRN internal representations

Once training is done, we want to examine the internal representations to see what the network has learned about the lexical items. Elman ran a hierarchical clustering analysis using the mean hidden representation of each word when presented across the corpus.

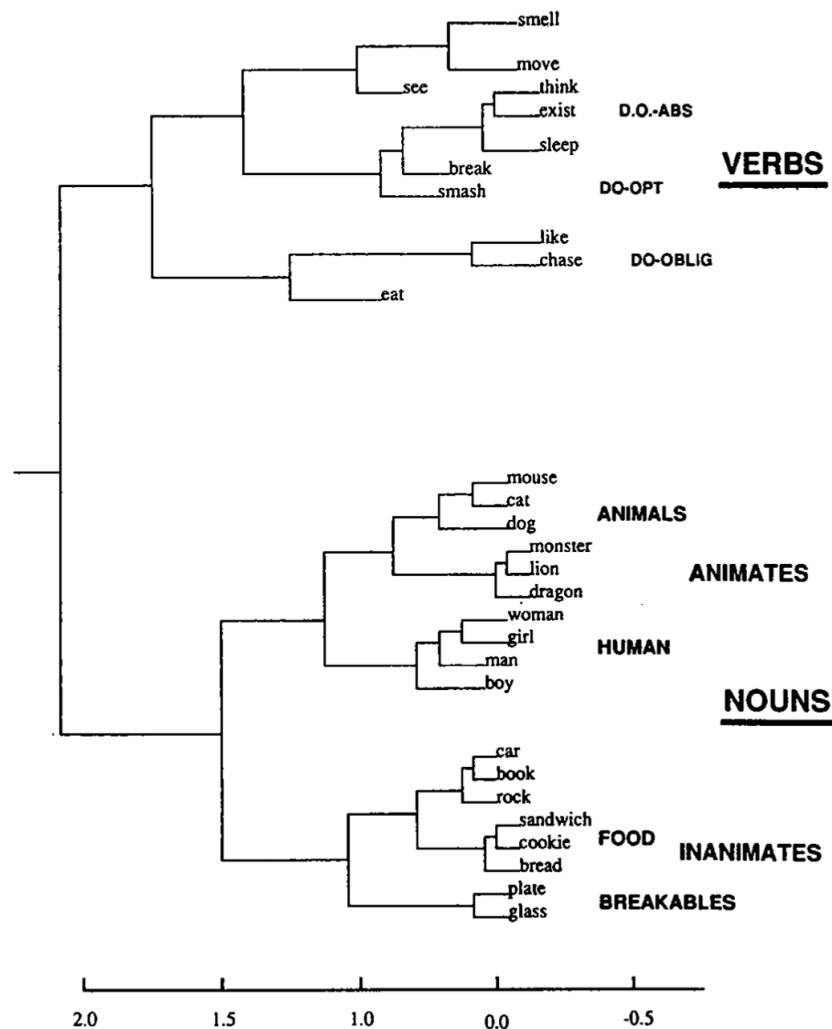
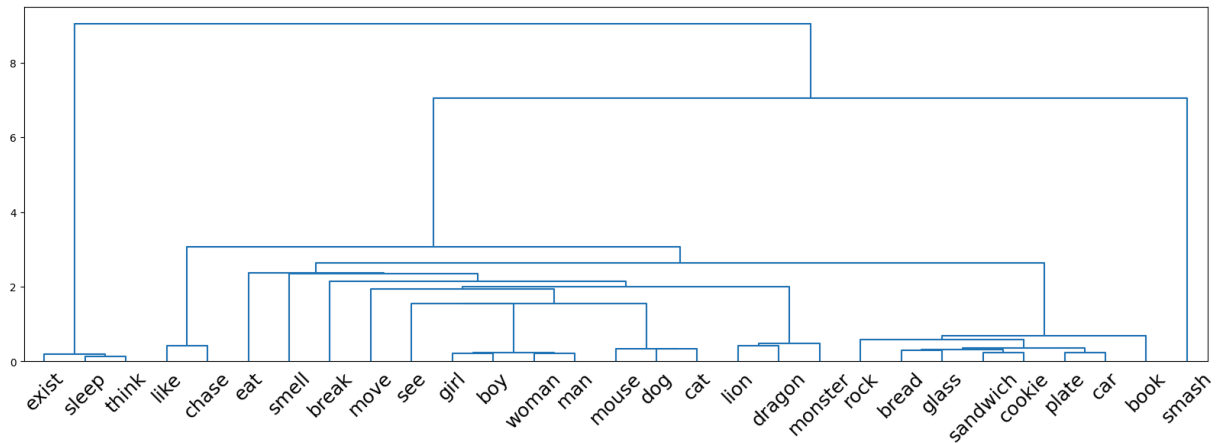


Figure 7. Hierarchical cluster diagram of hidden unit activation vectors in simple sentence prediction task. Labels indicate the inputs which produced the hidden unit vectors; inputs were presented in context, and the hidden unit vectors averaged across multiple contexts.

Unlike Elman we have an **explicit input embedding** for each word, and thus we can more simply look at these embedding vectors. Run the code to compare with Elman's results. *You shouldn't expect a close match.* There are differences in network architecture, training, and the dataset. Still, it's interesting to see what your SRN has learned.

```
In [6]: def plot_dendo(X, names, exclude=['<SOS>', '<EOS>']):
# Show hierarchical clustering of vectors
#
# Input
# X : numpy tensor [nitem x dim] such that each row is a vector to be c
# names : [length nitem] list of item names
# exclude: list of names we want to exclude
nitem = len(names)
names = np.array(names)
include = np.array([myname not in exclude for myname in names], dtype=bool)
linked = linkage(X[include], 'single', optimal_ordering=True)
plt.figure(1, figsize=(20,6))
dendrogram(linked, labels=names[include], color_threshold=0, leaf_font_s
plt.show()

plot_dendo(rnn.get_embeddings(), unique_tokens)
```



Problem 3 (10 points)

Write a function `generate` to probabilistically sample sentences from your network. Generate 10 sample sentences in this manner. For each, convert the sequence of token indices back to string form. When printing the sentence, you can either include the SOS and EOS or ignore them. It's fine to assume a maximum length.

Hint: You will find `torch.distributions.categorical.Categorical` useful.

```
In [7]: def generate(rnn, maxlen=4):
# TODO : YOUR CODE GOES HERE
```

```

with torch.no_grad():
    hidden = rnn.initHidden()
    token = token_to_index['<SOS>'] # start with <SOS> as the first input
    words = []
    for i in range(maxlen):
        output, hidden = rnn(torch.tensor(token), hidden)
        token = torch.distributions.categorical.Categorical(logits=output).sample()
        if index_to_token[token] == '<EOS>':
            break
        words.append(index_to_token[token])
    print(' '.join(words))

for i in range(10):
    generate(rnn)

```

girl see woman
 sandwich like
 dragon eat cat
 dragon eat bread
 monster eat lion
 girl see woman
 man eat cookie
 dog chase girl
 woman break plate
 monster break glass

Homework - Neural networks - Part F (30 points)

ALCOVE: An exemplar-based connectionist model of category learning

by Brenden Lake

In this part, you will help implement ALCOVE (Kruschke, 1992), an influential exemplar-based neural network model of human category learning. Unlike networks that learn abstract feature detectors in a hidden layer, ALCOVE stores every training item as an **exemplar** and classifies new items by their similarity to those stored exemplars. ALCOVE learns two things: how much attention to pay to each stimulus dimension, and how strongly each exemplar is associated with each category.

Kruschke, J. K. (1992). ALCOVE: An exemplar-based connectionist model of category learning. *Psychological Review*, 99(1), 22–44.

We will test ALCOVE on the classic benchmark from Shepard, Hovland, and Jenkins (1961; hereafter SHJ), a study that has influenced concept learning theory for over 60 years. SHJ used 8 stimuli that vary along 3 binary dimensions (e.g., shape, size, and color), and asked people to learn 6 different ways of sorting these 8 stimuli into two categories of 4. The 6 **problem types** differ in how many dimensions are relevant and how they interact:

- **Type I:** only 1 dimension is relevant (e.g., all category A items share one feature value).
- **Type II:** 2 dimensions are relevant, combined via XOR (this is the classic "exclusive or" problem).
- **Types III, IV, V:** 3 dimensions are relevant, but one dimension is *mostly* predictive with a few exceptions.
- **Type VI:** all 3 dimensions are relevant and none is more predictive than another (the hardest, most "arbitrary" partition).

People reliably learn these problems in the order $I < II < III \approx IV \approx V < VI$ (from easiest to hardest), even though every problem type asks for exactly the same number of stimulus-response pairs to be learned. Explaining this ordering is a classic challenge for models of category learning, and it is exactly what ALCOVE was built to address through its selective attention mechanism.

The 8 stimuli and 6 problem types

Each stimulus is a length-3 binary vector (one bit per dimension). The label files below give the category assignment ('A' or 'B') of each of the 8 stimuli, for each of the 6 problem types. Run the cell below to load and print them.

```
In [5]: # Packages we need
from __future__ import print_function
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib
%matplotlib inline
import matplotlib.pyplot as plt
```

```
In [6]: CLASS_A, CLASS_B = 1., 0.

def load_shj():
    # Load the 8 SHJ stimuli (3 binary dimensions each) and their category
    # labels for each of the 6 SHJ problem types.
    #
    # Output
    #   X : [8 x 3 [tensor] the 8 stimuli, one per row, used as ALCOVE's ex
    #   y_list : list of 6 [8 tensor], category label (1/0) for each stimulu
    #           one tensor per problem type
    stimuli = np.genfromtxt('data/shj_stimuli.txt', delimiter=',', dtype=flo
    labels = np.genfromtxt('data/shj_labels.txt', delimiter=',', dtype=str)
    X = torch.tensor(stimuli).float()
    y_list = []
    for mytype in range(labels.shape[0]):
        y = np.where(labels[mytype] == 'A', CLASS_A, CLASS_B)
        y_list.append(torch.tensor(y).float())
    return X, y_list

X, y_list = load_shj()
print('Stimuli (8 exemplars x 3 binary dimensions):')
print(X.numpy().astype(int))
print('')
for t in range(6):
    print('Type ' + str(t+1) + ' labels:', y_list[t].numpy())
```

Stimuli (8 exemplars x 3 binary dimensions):

```
[[0 0 0]
 [0 0 1]
 [0 1 0]
 [0 1 1]
 [1 0 0]
 [1 0 1]
 [1 1 0]
 [1 1 1]]
```

Type 1 labels: [1. 1. 1. 1. 0. 0. 0. 0.]

Type 2 labels: [1. 1. 0. 0. 0. 0. 1. 1.]

Type 3 labels: [0. 0. 0. 1. 1. 0. 1. 1.]

Type 4 labels: [0. 0. 0. 1. 0. 1. 1. 1.]

Type 5 labels: [0. 0. 0. 1. 1. 1. 1. 0.]

Type 6 labels: [0. 1. 1. 0. 1. 0. 0. 1.]

The ALCOVE model

ALCOVE represents a stimulus x by its similarity to every stored exemplar y_j (here, the 8 SHJ stimuli themselves). Similarity is governed by a city-block distance in psychological space, weighted by a set of learned attention weights α_i , one per stimulus dimension i :

$$\eta(x, y_j) = \exp\left(-c \sum_i \alpha_i |x_i - y_{ji}|\right)$$

Dimensions with a larger attention weight contribute more to similarity, so ALCOVE can learn to "tune out" irrelevant dimensions. The scalar c is a fixed specificity parameter that controls how quickly similarity falls off with distance.

Each exemplar j also has a learned association weight w_j to the category output. The network's similarity between stimulus x and category A is a weighted sum of exemplar activations:

$$\eta(x, A) = \sum_j w_j \eta(x, y_j)$$

Finally, a decision probability is obtained with a sigmoid (this is a simplification of Kruschke's original model, which uses a softmax over multiple categories and also incorporates $\eta(x, B)$, $\eta(x, C)$, etc.; here we have just 2 categories, so one logistic output suffices):

$$p(x \in A) = \text{logistic}(\phi \cdot \eta(x, A))$$

where ϕ is a fixed scaling parameter for the decision.

There are two sets of learnable parameters: the attention weights α_i (one per dimension) and the association weights w_j (one per exemplar). Both are learned by

gradient descent, but they use different learning rates, and the attention weights are constrained to stay non-negative.

Problem 1 (30 points)

Complete the `forward` method of the `ALCOVE` class below. Given a single stimulus `x` (a length-`dim` tensor), compute the probability of responding that `x` is in category `A`.

The constructor is already written for you: it stores the exemplars, initializes the attention weights to be uniform across dimensions, and initializes the association weights to zero.

Hint: `self.exemplars` is `[ne x dim]` and `x` is `[dim]`. You'll want to broadcast `x` against every row of `self.exemplars`, e.g. via `self.exemplars - x.view(1,-1)`, and similarly broadcast `self.attn` (shape `[dim]`) against the result.

```
In [7]: class ALCOVE(nn.Module):

    def __init__(self, exemplars, c=6.5, phi=2.5):
        # Input
        #   exemplars : [ne x dim tensor] rows are the exemplars stored in n
        #   c : specificity of the exemplar similarity gradient
        #   phi : decision temperature
        super().__init__()
        self.ne = exemplars.size(0) # number of exemplars
        self.dim = exemplars.size(1) # number of stimulus dimensions
        self.exemplars = exemplars

        # attention weights start out uniform across dimensions
        self.attn = nn.Parameter(torch.ones(self.dim) / float(self.dim))

        # association weights start out at zero (no basis yet for preferring
        self.w = nn.Linear(self.ne, 1, bias=False)
        self.w.weight = nn.Parameter(torch.zeros(1, self.ne))
        self.c = c
        self.phi = phi

    def forward(self, x):
        # Input
        #   x : [dim tensor] a single stimulus
        # Output
        #   output : [scalar tensor] unnormalized output (before the sigmoid
        #   prob : [scalar tensor] sigmoid decision probability

        # memory/hidden layer computes the similarity of stimulus x to each

        # YOUR CODE HERE
        hidden = torch.exp(-self.c * torch.sum(self.attn.view(1,-1) * torch.
```

```

        output = self.phi * self.w(hidden).squeeze() # phi * sum_j w_j * h_j
        prob = torch.sigmoid(output) # probability of category A
        return output, prob

```

Run the sanity check below on a freshly initialized network. Since the association weights start at zero, the output should be exactly 0 and the probability exactly 0.5 for every stimulus, regardless of the (uniform, untrained) attention weights.

```

In [8]: net = ALCOVE(X)
        print(net.ne)
        print('output, probability, for each stimulus:')
        for x in X:
            out, prob = net.forward(x)
            print(round(out.item(), 4), round(prob.item(), 4))

```

```

8
output, probability, for each stimulus:
0.0 0.5
0.0 0.5
0.0 0.5
0.0 0.5
0.0 0.5
0.0 0.5
0.0 0.5
0.0 0.5

```

Training ALCOVE

We train ALCOVE with **batch gradient descent**: on each epoch, the network sees all 8 exemplars, and a single optimizer step is taken using the summed loss across all of them. We use the standard **binary cross-entropy** loss on the network's raw output (before the sigmoid), via `torch.nn.BCEWithLogitsLoss` :

$$\ell(\text{output}, \text{target}) = -[\text{target} \cdot \log \text{logistic}(\text{output}) + (1 - \text{target}) \cdot \log(1 - \text{logistic}(\text{output}))]$$

where `target` is 1 for category A and 0 for category B. This is provided for you below, along with a `train` function and an `evaluate` function.

```

In [9]: def update_batch(net, exemplars, targets, loss, optimizer):
        # Update the weights using batch SGD for the entire set of exemplars
        #
        # Input
        #   exemplars: [ne x dim tensor] all stimuli/exemplars in experiment
        #   targets:   [ne tensor] classification targets (1./0.)
        #   loss:      function handle
        #   optimizer : SGD optimizer
        net.zero_grad()
        net.train()
        n_exemplars = exemplars.size(0)
        out = torch.zeros(n_exemplars)
        for j in range(n_exemplars):
            out[j], _ = net.forward(exemplars[j])

```

```

        myloss = loss(out, targets)
        myloss.backward()
        optimizer.step()
        net.attn.data = torch.clamp(net.attn.data, min=0.) # ensure attentio
        return myloss.cpu().item()

def train(exemplars, labels, num_epochs, lr_assoc=0.03, lr_attn=0.0033, track_inc=10, track_out=10):
    # Train an ALCOVE model on one SHJ problem
    #
    # Input
    #   exemplars : [ne x dim tensor] rows are exemplars
    #   labels : [ne tensor] category labels for this problem (1./0.)
    #   num_epochs : number of passes through the exemplar set
    #   lr_assoc : learning rate for association weights
    #   lr_attn : learning rate for attention weights
    #   track_inc : record progress every this many epochs
    # Output
    #   v_epoch, v_prob, v_acc : lists (same length) tracking epoch index,
    #                           probability correct, and percent accuracy
    n_exemplars = exemplars.size(0)
    net = ALCOVE(exemplars)
    optimizer = optim.SGD([
        {'params': net.w.parameters()},
        {'params': [net.attn], 'lr': lr_attn}
    ], lr=lr_assoc)
    loss = torch.nn.BCEWithLogitsLoss(reduction='sum')

    v_epoch = []
    v_prob = []
    v_acc = []
    for epoch in range(1, num_epochs + 1):
        loss_epoch = update_batch(net, exemplars, labels, loss, optimizer)
        if epoch == 1 or epoch % track_inc == 0:
            test_prob, test_acc = evaluate(net, exemplars, labels)
            v_epoch.append(epoch)
            v_prob.append(test_prob)
            v_acc.append(test_acc)

    return net, v_epoch, v_prob, v_acc

def evaluate(net, exemplars, targets):
    # Compute the mean probability of the correct response, and percent accuracy
    #
    # Input
    #   exemplars : [ne x dim tensor] all stimuli in the experiment
    #   targets : [ne tensor] classification targets (1./0.)
    net.eval()
    n_exemplars = exemplars.size(0)
    v_acc = np.zeros(n_exemplars)
    v_prob = np.zeros(n_exemplars)
    for j in range(n_exemplars):
        out, prob = net.forward(exemplars[j])
        out = out.item()
        prob = prob.item()
        if targets[j].item() == CLASS_A:
            v_prob[j] = prob

```

```

        v_acc[j] = out >= 0
    else:
        v_prob[j] = 1 - prob
        v_acc[j] = out < 0
    return np.mean(v_prob), 100. * np.mean(v_acc)

```

Run the cell below to train ALCOVE on all 6 SHJ problem types and plot the resulting learning curves. This should reproduce (approximately) the classic pattern of results from Kruschke (1992): Type I learned fastest, Type VI learned slowest, with Types III–V in between. **Note that this pattern will not be exactly as in the slides, which used Kruschke's original loss function rather than binary cross entropy.**

```

In [10]: num_epochs = 50
list_nets = []
list_trackers = []
for mytype in range(6):
    print('Training on Type ' + str(mytype + 1) + '...')
    net, v_epoch, v_prob, v_acc = train(X, y_list[mytype], num_epochs)
    list_nets.append(net)
    list_trackers.append((v_epoch, 1-np.array(v_prob), v_acc))

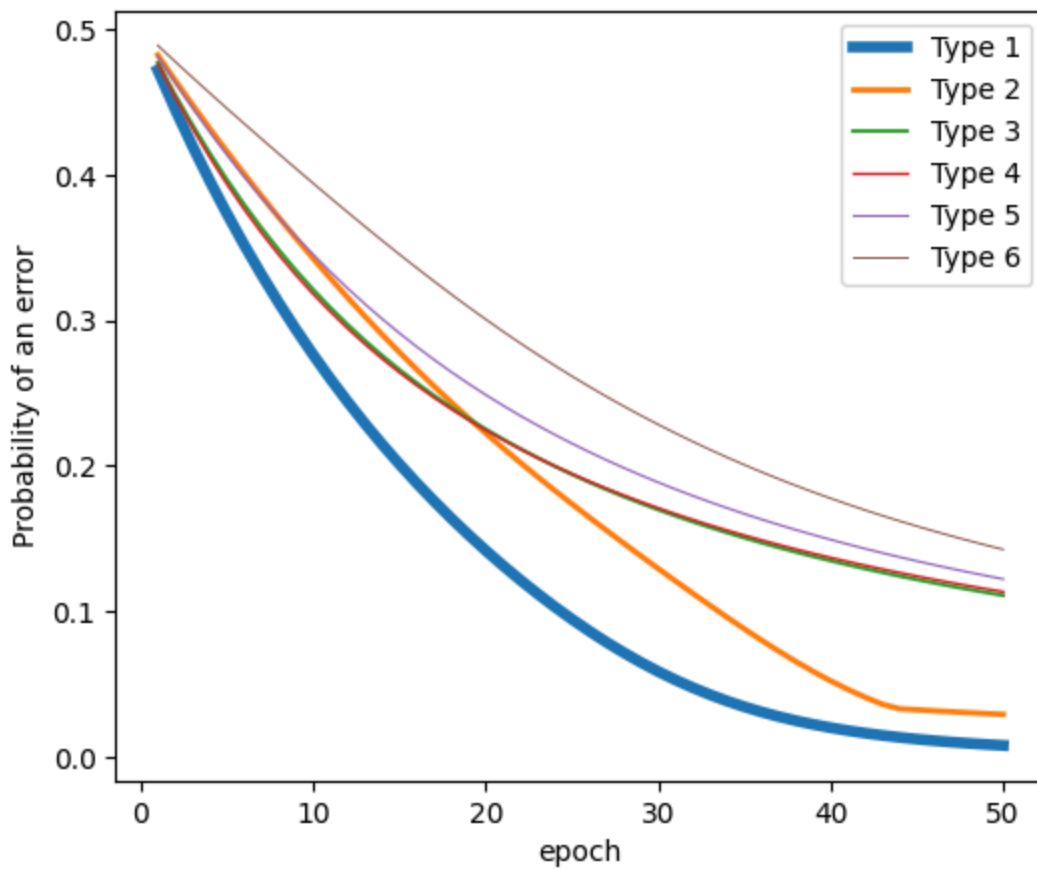
plt.figure(figsize=(6, 5))
for mytype in range(6):
    v_epoch, v_prob, v_acc = list_trackers[mytype]
    plt.plot(v_epoch, v_prob, linewidth=4. / (mytype + 1), label='Type ' + str(mytype + 1))
plt.xlabel('epoch')
plt.ylabel('Probability of an error')
plt.legend()
plt.show()

```

```

Training on Type 1...
Training on Type 2...
Training on Type 3...
Training on Type 4...
Training on Type 5...
Training on Type 6...

```



Here is what my learning curve looked like:

